Assignment: Ad-Enabled LLM as a Service

Course Project (Topic 8)

by

Name	Matric No.	Work %
Timothy Chang Kai En	*U2220136J*	50.0%
Joel Lee Pak Xin	*UXXXXXXX*	50.0%

SC4052 Cloud Computing

NANYANG TECHNOLOGICAL UNIVERSITY

April 2026

Abstract

Contents

List of Figures

List of Tables

List of Algorithms

1 Introduction

1.1 Background and Motivation

The dominant paradigm in digital advertising has, for two decades, relied on a fundamental assumption: the user interacts with a visual interface—a web page, a search results listing, a social media feed—where rectangular display regions can be allocated to sponsored content. Banner advertisements, programmatic display auctions, and search-engine sponsored links all exploit this property. The advertiser purchases screen real estate; the publisher renders a creative unit; the user either attends to it or does not. This model, refined through successive generations of behavioural targeting and real-time bidding, generates hundreds of billions of dollars in annual revenue worldwide and underpins the economic viability of most free-to-use internet services.

Conversational AI interfaces disrupt this assumption at its foundation. Systems such as ChatGPT, Claude, and Gemini present information as a single stream of natural-language text. There are no sidebars, no interstitial slots, and no visual regions in which a banner can be placed without breaking the conversational contract between the system and its user. At the same time, these interfaces are absorbing an increasing share of the queries that users previously directed to search engines—product research, comparison shopping, recommendation requests—precisely the query types that carry the highest commercial intent and, consequently, the highest advertising value. The shift is not merely a change of medium; it is a structural transformation in how users express intent, receive information, and make purchasing decisions.

The advertising industry therefore faces a concrete technical problem: how to monetise conversational AI interactions without degrading the user experience that makes them valuable in the first place. Appending a block of sponsored links to the end of a chat response replicates the search-ad format but sacrifices the natural feel of the conversation. Inserting interstitial advertisements between turns interrupts the dialogue flow and erodes user trust. Neither approach exploits the distinctive capability of a large language model (LLM), namely its ability to generate contextually appropriate, fluent natural-language text that integrates information from multiple sources—including sponsored product information—into a single coherent response.

This observation motivates the present work. If the LLM can understand the user's intent, match it against an inventory of relevant products, and weave a sponsored recommendation naturally into an otherwise helpful response, then the advertisement ceases to be an interruption and becomes, instead, part of the answer. The result is an advertising format that is native to the conversational medium: contextually relevant, linguistically integrated, and measurable not merely through impressions and clicks but through the user's subsequent conversational engagement with the recommended product.

The same logic extends beyond text. Conversational AI interfaces increasingly support image generation as a native modality, and a user who asks the system to render a scene is expressing creative and often commercial intent in a form that is no less monetisable than a textual product query. If the language model can weave a sponsored product into a written answer, an image model can, by analogous reasoning, place that product naturally within a generated scene—on a shelf, in a character's hand, within the environment the user described—rather than overlaying a banner on top of the artwork. Treating text and image generation as two surfaces of a single advertising pipeline is therefore a natural extension of the core thesis, and one that this project pursues explicitly.

A workable platform, however, cannot consist of the generative pipeline alone. A production-grade advertising system also requires the operational scaffolding that makes it usable by real advertisers: a way to onboard them, a way to organise their spend into campaigns with start and end dates and daily pacing, a way to create and manage ads without touching the database directly, and a way for both the platform operator and the advertiser to observe how their money is being spent. These concerns are conventionally relegated to "future work," but they fundamentally shape the data model and the request path—budget gating, for example, must happen inside the ad-retrieval loop, not as an afterthought—and this project therefore treats them as first-class deliverables rather than deferred extensions.

1.2 Project Objectives

This project designs, implements, and evaluates an end-to-end LLM-powered conversational advertising platform—referred to throughout this report as *Ad LLM as a Service*—that delivers inline sponsored product recommendations within AI chat responses. The platform satisfies the following objectives:

(i) **Intent-aware context extraction.** A lightweight LLM call extracts structured context from each user message and the preceding conversation history, producing a classification of intent, topical relevance, sentiment, purchase signal strength, and—critically—a sensitivity-aware ad receptivity score that gates whether any advertisement should be shown at all.

(ii) **Semantic ad matching with budget-aware eligibility.** The extracted context is embedded into a vector representation and matched against a pre-embedded ad inventory via cosine similarity search over `pgvector`. Candidate ads are then filtered through a campaign eligibility check—verifying that the parent campaign is active, within its scheduled window, and has not exhausted either its total budget or its daily cap—and re-ranked by a business-rule stage that incorporates bid price, per-session frequency caps, and brand-safety tags.

(iii) **Natural ad integration with mandatory disclosure.** The main response-generation LLM receives the matched advertisement as part of its prompt context and is instructed to weave the product mention into a helpful answer if and only if the product is genuinely relevant. Every sponsored mention carries a `[Sponsored]` disclosure tag, ensuring compliance with Federal Trade Commission (FTC) endorsement guidelines.

(iv) **Multi-dimensional response and engagement evaluation.** Beyond conventional impression and click-through tracking, a dedicated LLM-as-judge module analyses each post-ad follow-up message and returns a structured set of metrics: an engagement type classification (product inquiry, comparison, price inquiry, purchase intent, feature question, positive or negative reaction, dismissal), a continuous engagement score, sentiment toward the advertisement, a naturalness score capturing how intrusive the placement felt, and an estimated purchase-proximity score. These metrics jointly form a higher-fidelity signal than any single click-or-not-click measurement and are the substrate on which downstream optimisation operates.

(v) **Ethical safeguards.** Conversations classified as sensitive—those involving health crises, emotional distress, financial hardship, or grief—are automatically excluded from ad injection, regardless of commercial relevance.

(vi) **Adaptive per-ad context optimisation.** Each advertisement in the inventory carries a dynamic presentation-guidance document—its *ad context*—which is injected into the response-synthesis prompt alongside the product description. An initial version of this context is generated automatically by an LLM at the moment the ad is created, so that every ad begins life with a coherent presentation strategy rather than an empty field. A background optimiser then periodically reviews the accumulated engagement metrics for each ad, diagnoses what is working and what is not (low naturalness, high dismissal rate, weak purchase proximity despite strong topical match), and asks a reasoning LLM to rewrite the context guide accordingly. Every revision is versioned and persisted to an `ad_context_history` table together with the metrics snapshot that triggered it and a natural-language justification, yielding a fully auditable record of how each ad's presentation strategy evolved over time.

(vii) **Image generation with organic product placement.** The platform exposes a second generative surface alongside text chat: a user-prompted image-generation endpoint that routes through the same context extraction and ad matching pipeline. When a relevant sponsored product is identified, the user's original image prompt is rewritten by an auxiliary LLM into a placement prompt that fits the product into the requested scene, and the final image is rendered by a diffusion model (`gpt-image-1`). When the advertised product has a reference photograph, it is supplied to the image model via an edit-mode call so that the rendered product is visually grounded in the real article rather than hallucinated from its textual description.

(viii) **Advertiser portal with campaign and budget management.** A self-serve REST API and accompanying React interface allow advertisers to register, organise their spend into campaigns with optional start dates, end dates, and daily budget caps, and create ads within each campaign. Campaigns follow an explicit lifecycle—`draft`, `active`, `paused`, `completed`—with auto-completion when the total budget is exhausted, and daily spend is reset lazily at first touch each new day rather than via a cron job. Ad creation is accelerated by a URL-based extraction flow in which the advertiser pastes a product page URL, the server scrapes it, and an LLM call populates the ad fields (name, description, creative copy, target topics and intents, brand-safety tags) as a pre-filled draft that the advertiser can review and edit before submission.

(ix) **Dual analytics dashboards.** The platform ships two distinct analytics surfaces backed by a shared event log. A global, platform-wide dashboard reports aggregate impressions, click-through rate, engagement events, estimated revenue, and top-performing ads across all advertisers, intended for the platform operator. An advertiser-facing dashboard reports the same metrics scoped to a single advertiser or a single campaign, with a daily breakdown driven by the `daily_spend_log` table and visual indicators of budget utilisation.

(x) **Functional MVP.** The deliverable is a working prototype that demonstrates the complete pipeline from user message to ad-augmented response—across both text and image modalities—together with the advertiser portal, the adaptive context optimiser, and both analytics dashboards.

1.3 Scope and Constraints

The platform is scoped as a minimum viable product (MVP) and is subject to several deliberate constraints that bound the design space.

The architecture is a single-service deployment: one FastAPI orchestrator coordinates all internal modules, a single Redis instance manages session state, and a single PostgreSQL database (extended with `pgvector`) stores the ad inventory, the advertiser and campaign records, the daily spend log, the ad-context history, and the event log. This monolithic design is appropriate for the current scale—tens of thousands of ads, moderate request concurrency—and avoids the operational complexity of a distributed microservice architecture.

Two external dependencies impose hard constraints on latency and cost. Every chat request requires at minimum two LLM API calls—one lightweight call for context extraction and one full-model call for response synthesis—plus one embedding API call for the query vector. The total end-to-end latency budget is three seconds, inclusive of all network round-trips and internal processing. The per-request cost is dominated by these API calls, and the re-ranking stage uses heuristic scoring weights (a weighted combination of semantic similarity, normalised bid price, and purchase signal) rather than a learned model, since no click-through data is available at launch. Image generation is treated as a separate interactive surface with its own, looser latency budget, since diffusion-model calls routinely exceed ten seconds and are dominated by GPU inference time rather than by the orchestration overhead that constrains the text path.

The engagement evaluator is implemented as a single LLM call that returns the structured metrics described in Section 1.2. It runs asynchronously, off the critical request path, so its latency does not count against the three-second response budget; the trade-off is that engagement metrics become available only after a short delay rather than synchronously with the turn in which the ad was shown.

The adaptive context optimiser is similarly a deferred, out-of-band process. Optimisation for a given advertisement is gated by two guards—a minimum of five recorded engagements and a minimum interval of twenty-four hours since the previous revision—which together prevent the system from overfitting to noise and from thrashing the ad context under light traffic. An advertisement with insufficient traffic therefore retains its LLM-generated initial context indefinitely, and optimisation only begins once the statistical foundation is adequate.

The advertiser portal is scoped to the operational primitives needed to run a campaign end-to-end: advertiser registration, campaign CRUD with budget and schedule controls, ad CRUD with URL-based pre-filling, and scoped analytics. Authentication middleware, invoicing, payment processing, and advertiser self-service account recovery are deliberately out of scope for the MVP; the portal assumes a trusted operator environment and defers commercial-grade identity and billing to subsequent iterations.

1.4 Report Organisation

The remainder of this report is organised as follows. Section 2 surveys the relevant literature across four areas: the evolution of digital advertising formats, the capabilities of large language models for controlled generation, retrieval-augmented generation as a paradigm for context-aware information retrieval, and the emerging body of work on advertising within conversational interfaces. Section 4 formulates the problem precisely, identifying the technical challenges of real-time intent extraction, low-latency semantic ad matching, natural ad integration, engagement measurement, and per-campaign budget enforcement, together with the ethical and regulatory constraints that any solution must satisfy. Section 5 presents the system architecture and the design of each component: the FastAPI orchestrator, the context extraction module, the Redis-backed session manager, the `pgvector`-based ad matching pipeline with budget-gated eligibility, the prompt-engineered response synthesis module, the image-generation endpoint with product-placement prompt rewriting, the LLM-as-judge engagement evaluator, the adaptive ad-context optimiser, the advertiser portal with its URL-based ad extraction flow, and the dual analytics subsystem serving both the platform-wide and advertiser-scoped dashboards. Section **??** describes the implementation in detail, including workstream responsibilities, the database schema, API contracts, and the quality assurance framework. Section 6 walks through end-to-end examples that exercise representative paths through the pipeline, including successful ad injection, sensitive-conversation suppression, irrelevant-ad rejection, engagement follow-up detection, multi-turn context accumulation, an image-generation request with organic product placement, a campaign reaching its daily budget cap, and an advertiser creating a new ad via URL extraction. Section 7 summarises the contributions, discusses limitations, and outlines directions for future work.

2 Literature Review

This section surveys the bodies of work that inform the design of the *Ad LLM as a Service* platform. The review is organised by *approach* rather than by chronology: each subsection isolates a distinct technical or methodological strand—digital advertising formats, large language model capabilities, retrieval-augmented generation, conversational-interface monetisation, prompt engineering for controlled output, and engagement measurement—and positions the present project relative to the state of the art in that strand. Section 2.9 synthesises the findings and identifies the specific research gap that this project addresses.

2.1 Evolution of Digital Advertising

The economic architecture of online advertising has undergone three major structural transitions since the first banner advertisement appeared on HotWired in 1994, an AT&T creative unit that reportedly achieved a click-through rate exceeding 40% [20].

From display to programmatic. The first generation of digital advertising was transactional and manual: publishers sold rectangular display regions at fixed rates per thousand impressions, and advertisers negotiated placements directly with individual sites. The introduction of ad servers—most notably DoubleClick in 1996—centralised delivery and measurement, but the buying process remained fundamentally a media-planning exercise. The emergence of real-time bidding (RTB) between 2007 and 2010 transformed this model into an auction-based marketplace in which individual impressions are priced algorithmically on the basis of predicted user value [47]. Demand-side platforms (DSPs), supply-side platforms (SSPs), and data management platforms (DMPs) together created a programmatic stack that, by 2022, accounted for approximately 84% of global digital display spending [12].

Search advertising and intent-based targeting. Google's introduction of AdWords in 2000, later renamed Google Ads, established a second paradigm: intent-based targeting. Rather than inferring user interest from page context or behavioural profiles, search advertising matches sponsored results directly to the user's expressed query. This alignment of commercial supply with declared demand yields conversion rates substantially higher than those of display advertising and has made search the single largest category of digital ad spending. In 2024, Google's total advertising revenue reached approximately $265 billion, with the majority derived from search placements [40]. The economic logic is straightforward: a user who searches for "best running shoes for flat feet" has explicitly declared a purchase-adjacent intent that no amount of behavioural inference on a display network can reliably reconstruct.

Social and behavioural targeting. A third strand developed in parallel on social-media platforms, where dense self-reported demographic data and interaction graphs enabled highly granular audience segmentation. Facebook (now Meta), Instagram, and TikTok built advertising businesses on the principle that a user's social graph, expressed interests, and engagement patterns constitute a rich behavioural profile against which ads can be targeted even in the absence of an explicit query. This model trades the precision of declared intent for the breadth of passive behavioural signal and has proven effective for awareness and consideration campaigns, though it depends on third-party cookies and device identifiers that are now being phased out under privacy regulations such as the General Data Protection Regulation (GDPR) and Apple's App Tracking Transparency framework [2], [13].

The move toward native formats. Across all three paradigms, a persistent trend has been the migration from visually distinct ad units—banners, interstitials, pop-ups—toward formats that are structurally integrated with the surrounding content. Native advertising, defined as paid content that matches the form and function of the host medium, arose as a response to banner blindness and ad-blocker adoption [45]. Sponsored articles on news sites, promoted posts on social feeds, and influencer endorsements on video platforms all share the property that the advertisement is embedded within the content stream rather than placed adjacent to it. The platform examined in this report extends this principle to its logical terminus in conversational AI: the sponsored recommendation is not merely placed alongside the response but woven into it by the language model itself.

2.2 Large Language Models and Conversational AI

The transformer architecture. The architectural foundation of the present system is the transformer, introduced by Vaswani et al. [42]. The transformer replaces the recurrent processing of earlier sequence-to-sequence models with a self-attention mechanism that computes pairwise relevance scores across all positions in the input simultaneously, enabling parallelised training and effective capture of long-range dependencies. This architecture has become the dominant substrate for language modelling: every major large language model (LLM) deployed as of 2025—including OpenAI's GPT series [7], [31], Anthropic's Claude family [1], Google's Gemini [18], and Meta's LLaMA [41]—is built on transformer variants. The progression from GPT-2's 117 million parameters to GPT-4's estimated hundreds of billions illustrates the scaling trajectory that has yielded the emergent capabilities exploited in this project [44].

Capabilities relevant to this project. Four capabilities of modern LLMs are directly exercised by the Ad LLM platform:

(i) *Instruction following.* Post-training alignment techniques—reinforcement learning from human feedback (RLHF) [32] and constitutional AI [3]—have produced models that reliably follow detailed natural-language instructions, including multi-constraint prompts that specify output format, tone, and content restrictions.

(ii) *Structured output generation.* When instructed appropriately, LLMs generate syntactically valid JSON, markdown, or other structured formats with high consistency. This capability underpins the context extraction module, which must return a machine-parseable classification of intent, topics, sentiment, and ad receptivity from each user message.

(iii) *Contextual reasoning over long inputs.* Context windows of 100 000 tokens and above—available in models such as Claude 3 (200 000 tokens) and Gemini 1.5 (up to 1 000 000 tokens) [1], [18]—allow the model to condition its generation on extensive conversation histories, a prerequisite for maintaining coherent multi-turn advertising integration.

(iv) *Tone adaptation.* LLMs modulate register, formality, and lexical choice in response to system-prompt instructions, enabling the response synthesis module to match the conversational style of the user while integrating a sponsored product mention.

Lightweight versus capable models. A practical distinction that shapes the system architecture is the trade-off between model capability and inference cost. Lightweight models—exemplified by Anthropic's Haiku tier—offer low latency and low per-token cost at the expense of reduced reasoning depth. Capable models—exemplified by the Sonnet tier—produce higher-quality, more nuanced text but at greater cost and latency. The Ad LLM platform exploits this spectrum deliberately: context extraction, a classification task requiring structured output but not creative generation, is assigned to a Haiku-class model; response synthesis, which demands natural integration of a product mention into a helpful answer, is assigned to a Sonnet-class model [1]. This two-tier strategy is consistent with the broader industry practice of routing sub-tasks to the smallest model that achieves acceptable quality, thereby minimising aggregate inference cost [10].

2.3 Retrieval-Augmented Generation (RAG)

The RAG paradigm. Retrieval-augmented generation, formalised by Lewis et al. [25], addresses a fundamental limitation of parametric language models: their knowledge is frozen at training time and cannot be updated or extended without retraining. The RAG architecture augments the generative model with a non-parametric memory—a corpus of documents indexed for efficient retrieval—so that, at inference time, the model conditions its output on both the user query and the most relevant retrieved passages. Lewis et al. demonstrated that this combination of a pre-trained seq2seq transformer (BART) with a dense vector index of Wikipedia, accessed via a neural retriever (Dense Passage Retriever, DPR) [23], achieved state-of-the-art results on open-domain question answering benchmarks and generated more factual, specific language than parametric-only baselines.

Vector embeddings and semantic similarity. The retrieval component of a RAG pipeline depends on dense vector representations of both the query and the candidate documents.

Embedding models—such as OpenAI's `text-embedding-3-small`, Cohere's Embed, and open-source alternatives like BGE and E5 [43], [46]—map text into high-dimensional vector spaces in which semantic similarity corresponds to geometric proximity (typically measured by cosine distance). The quality of these embeddings determines retrieval precision: a query embedding that accurately captures the user's informational need will surface documents whose content is genuinely relevant, while a poor embedding will introduce noise that degrades the generator's output.

Vector databases and extensions. At scale, approximate nearest-neighbour (ANN) search over embedding vectors requires specialised indexing. Dedicated vector databases such as Pinecone, Weaviate, and Milvus provide managed infrastructure for this purpose [21]. For applications with moderate inventory sizes—tens of thousands of records rather than billions—the `pgvector` extension to PostgreSQL offers a pragmatic alternative: it adds vector-typed columns and cosine-distance operators to a relational database, avoiding the operational overhead of a separate vector store [24]. The Ad LLM platform adopts `pgvector` for precisely this reason: the ad inventory is expected to contain at most tens of thousands of items, and co-locating vector search with the relational schema for budget management, frequency capping, and event logging simplifies both development and deployment.

Ad matching as a specialised RAG pipeline. The conceptual contribution of the present project's ad-matching subsystem is the reinterpretation of RAG in a commercial context. In a conventional RAG system, the retrieval step fetches passages that are informationally relevant to the user's question; in the Ad LLM pipeline, the retrieval step fetches *advertisements* that are semantically relevant to the user's conversational context. The query embedding is constructed not from the raw user message but from the structured context extracted by the lightweight LLM—topics, intent, and entities—ensuring that the retrieval signal reflects interpreted user need rather than surface lexical overlap. A business-rule re-ranking stage then applies non-semantic constraints (budget availability, bid price, per-session frequency caps) before the top candidates are passed to the generator. This architecture preserves the retrieve-then-generate structure of RAG while specialising both the retrieval objective and the post-retrieval filtering to the requirements of advertising.

2.4 Advertising in Conversational Interfaces

Industry deployments. The integration of advertising into conversational AI is a recent and rapidly evolving development. Microsoft was the first major platform to insert sponsored content into a conversational interface, embedding text ads, shopping campaigns, and multimedia placements into Bing Chat responses shortly after its launch in February 2023 [29]. In September 2023, Microsoft announced "Conversational Ads," a new format designed specifically for the chat medium, including "Compare & Decide Ads" that present structured product comparisons within the dialogue flow [28]. Google followed with experiments in its Search Generative Experience (SGE), placing sponsored results both adjacent to and within AI-generated answer summaries, labelled with a bold "Sponsored" tag [16]. By 2025, Google had extended these placements into AI Mode, a fully conversational search interface powered by Gemini 2.0, and reported that ads drawn from existing Search and Shopping campaigns were being served within AI-generated responses [17].

These deployments share a common limitation from the perspective of the present project: they append or interleave conventional ad units (text links, product cards) alongside AI-generated text rather than instructing the language model to integrate the product mention *into* the response itself. The result is a hybrid layout in which the advertisement remains visually and linguistically distinct from the organic answer—a format that is arguably native to the search-results page but not to the conversational medium.

Academic work on native advertising and user perception. A substantial body of research examines how users perceive and respond to native advertising. Wojdynski and Evans [45] demonstrated that fewer than one in ten participants recognised article-style native advertising as paid content, even when disclosure labels were present, highlighting the tension between format integration and consumer awareness. Subsequent work has shown that this tension operates through two competing mechanisms [11]: explicit sponsorship disclosures activate persuasion knowledge, prompting critical evaluation and reducing brand attitude, but simultaneously enhance perceived transparency, which increases trust and partially offsets the negative effect. The balance between these mechanisms depends on disclosure language, placement, and prominence [8], [45]. A Nielsen study reported that 79% of users correctly identified content as advertising when the label read "Advertisement," compared to only 48% for the label "Presented by" [30]. These findings directly inform the design decision in the Ad LLM platform to use a mandatory, visually rendered `[Sponsored]` tag—a disclosure format that is both explicit and machine-enforceable.

Ethical considerations: transparency and trust. The ethical literature on advertising transparency converges on a central finding: consumers respond more favourably to advertising when they perceive that the advertiser is being honest about the commercial nature of the content [8], [9]. Transparency functions as a signal of brand integrity [22], and the downstream effects on trust and purchase intention are robust across experimental replications. In the conversational AI context, the stakes are amplified: the user has entered a dialogue that they expect to be helpful and impartial, and undisclosed commercial influence would constitute a violation of that implicit contract. The Ad LLM platform addresses this risk through three mechanisms: (i) the `[Sponsored]` tag is mandatory and enforced at the prompt level; (ii) the response-generation prompt instructs the model to ignore the advertisement if it is not relevant to the user's question; and (iii) conversations classified as sensitive are automatically excluded from ad injection; and (iv) each ad carries brand-safety tags that allow advertisers to declare topics their creative must not appear alongside, providing a second, advertiser-controlled filter layered on top of the platform's sensitivity gate.

Regulatory frameworks. The Federal Trade Commission (FTC) revised its Endorsement Guides in July 2023, updating the definition of "endorsement" to encompass digital formats including social-media tags, virtual influencers, and AI-generated recommendations [14]. The revised Guides require that any material connection between an advertiser and an endorser be disclosed "clearly and conspicuously," using language that is difficult to miss and easily understood by the target audience. The Guides explicitly note that platform-provided disclosure tools (e.g., Instagram's "Paid partnership" label) may not be sufficient on their own. Although the FTC has not yet issued guidance specific to LLM-generated sponsored content, the general principle—that consumers must be informed when content is commercially motivated—applies directly. The `[Sponsored]` disclosure mechanism in the Ad LLM platform is designed to satisfy this principle by ensuring that every sponsored product mention is preceded by an unambiguous, machine-enforced marker.

2.5 Prompt Engineering for Controlled Generation

Techniques for reliable structured output. Prompt engineering—the practice of crafting input instructions to elicit desired model behaviour without modifying model parameters—has emerged as a critical discipline in applied LLM systems. Schulhoff et al. [38] present a taxonomy of 58 distinct prompting techniques, ranging from zero-shot and few-shot prompting to chain-of-thought reasoning, self-refinement, and retrieval-augmented prompting. Sahoo et al. [37] provide a complementary survey organised by application area, demonstrating that prompt design significantly affects output quality across tasks including question answering, code generation, and text classification. For the Ad LLM platform, the most relevant techniques are *role assignment* (instructing the model to adopt a specific persona), *output-format specification* (requiring JSON or markdown structure), and *conditional instruction* (directing the model to include or exclude content based on stated criteria).

System prompts and instruction following. The system prompt—a privileged instruction block that precedes the user's message and persists across turns—is the primary mechanism through which the Ad LLM platform controls the response-generation model. The system prompt specifies: the model's role (a helpful assistant), the conditions under which a sponsored product should be mentioned (relevance to the user's question), the required disclosure format (`[Sponsored]` tag), and the link format (markdown with tracking URL). Ouyang et al. [32] showed that instruction-tuned models follow system-level directives with high fidelity, though compliance is not absolute: adversarial or ambiguous inputs can cause the model to deviate from instructions. The prompt design for the Ad LLM platform therefore includes explicit rejection logic ("If the product is not relevant to the conversation, ignore it entirely") and is validated against an evaluation set of 20–30 scenarios covering both successful integration and correct rejection.

Balancing helpfulness with ad integration naturalness. A distinctive prompt-engineering challenge in this project is the dual-objective nature of the generation task: the response must simultaneously be *helpful* (answering the user's question accurately and completely) and *commercially effective* (weaving a product mention into the answer in a way that feels like an informed suggestion rather than an insertion). These objectives are not inherently conflicting—a relevant product recommendation *is* helpful—but they become so when the matched advertisement is only tangentially related to the user's query. The prompt must therefore encode a threshold: the model should integrate the product only when doing so genuinely enhances the response, and should produce a purely organic answer otherwise. Achieving consistent behaviour across diverse conversation types requires iterative prompt refinement guided by systematic evaluation.

LLM-as-judge for automated quality evaluation. Manual evaluation of generated responses does not scale. The LLM-as-judge paradigm, surveyed by Zheng et al. [48] and Gu et al. [19], addresses this limitation by using a separate LLM call to score generated outputs along specified quality dimensions. Zheng et al. demonstrated that GPT-4, when used as a judge, achieves approximately 80% agreement with human preference scores—matching the inter-annotator agreement rate among human raters themselves. The Ad LLM platform adopts this paradigm for offline quality monitoring: a judge model scores sampled production responses on helpfulness, naturalness of ad integration, disclosure accuracy, and ad relevance, enabling automated detection of quality regressions without requiring continuous human review.

Beyond sampled offline monitoring, the Ad LLM platform extends the LLM-as-judge paradigm into an online measurement role: every follow-up turn after an ad impression is passed to a judge model that returns a structured multi-dimensional assessment comprising an engagement score, an ad-naturalness score, an estimated purchase-proximity score, a sentiment polarity, and a categorical engagement type (e.g., *product inquiry*, *comparison*, *price inquiry*, *purchase intent*, *dismissal*). This contrasts with most prior LLM-as-judge deployments, which treat the judge as an offline evaluator of a held-out test set. Using the judge as an *in-the-loop* instrument raises new considerations—judge consistency across runs, the cost of an additional LLM call per engaged turn, and the risk of judge drift as the underlying model is updated—that are under-explored in the current literature and which this project confronts directly.

2.6 User Engagement Metrics in Conversational Systems

Traditional ad metrics. The standard measurement framework for digital advertising centres on three quantities: impressions (the number of times an ad is rendered), click-through rate (CTR, the fraction of impressions that result in a click), and conversion rate (the fraction of clicks that result in a desired action, such as a purchase). These metrics are well-understood and form the basis of pricing models including cost per mille (CPM) and cost per click (CPC). However, they were designed for a visual medium in which the user's attention is inferred from the rendering of a display unit. In a conversational interface, where the entire response is a continuous text stream, the concept of a discrete "impression" is less well defined, and a "click" on an embedded link may not capture the full spectrum of user engagement with the recommended product.

A multi-dimensional engagement vector. In practice the Ad LLM platform decomposes "engagement" into a vector of six signals produced by a judge model on each post-ad follow-up turn: (a) a binary engaged/not-engaged flag; (b) an *engagement type* drawn from a fixed taxonomy (product inquiry, comparison, price inquiry, purchase intent, feature question, positive reaction, negative reaction, dismissal); (c) a continuous engagement score in $[0, 1]$; (d) an ad-naturalness score in $[0, 1]$ estimating how organic the placement felt from the user's reaction; (e) a purchase-proximity score in $[0, 1]$ approximating how close the user appears to a buying decision; and (f) a sentiment polarity toward the ad. This decomposition is more informative than a single scalar engagement value because it separates *whether* the user engaged from *how* they engaged and *how the ad was received*, enabling the downstream optimisation described in Section 2.7 below. It also aligns with the direction advocated by Mehri and Eskénazi [27], who argue that dialogue quality is inherently multi-faceted and resists collapse to a single number.

Why conversational engagement is a stronger signal. The fundamental advantage of conversational engagement metrics over traditional click-through measurement is that they capture *active cognitive processing* rather than a binary click-or-not-click signal. A user who asks a follow-up question about an advertised product has read the recommendation, understood it, found it relevant, and invested the effort to formulate a new query. This chain of cognitive steps is substantially more informative than a click, which may be accidental, curiosity-driven, or immediately followed by a bounce. Conversational engagement therefore offers advertisers a higher-fidelity signal of genuine interest, which in turn supports more accurate attribution and, ultimately, more efficient allocation of advertising spend.

Related work on measuring user attention in dialogue. Research on dialogue systems has developed several proxies for user engagement, including turn length, response latency, sentiment trajectory, and topic persistence [27], [39]. See et al. [39] found that longer user responses and sustained topic continuity correlate with self-reported satisfaction in open-domain dialogue. Mehri and Eskénazi [27] proposed unsupervised dialogue quality metrics based on response coherence and relevance scoring. The engagement classifier in the Ad LLM platform draws on this work by treating topic persistence (continued discussion of the advertised product) as one component of engagement, but goes beyond lexical proxies by delegating the full classification to an LLM judge that returns the structured six-signal vector described above. This replaces an earlier keyword-and-entity-matching prototype, and is made tractable by the same LLM-as-judge infrastructure used for offline quality monitoring.

2.7 Adaptive Prompt and Context Optimisation

From static prompts to data-driven prompt refinement. The prompt-engineering literature surveyed in Section 2.5 treats the system prompt as a human-authored artefact that is iterated manually against an evaluation set. A growing body of work has begun to automate this loop. Automatic Prompt Engineer (APE) [49] frames prompt design as a black-box search problem in which candidate instructions are generated by an LLM, scored against a task metric, and resampled. PromptBreeder [15] extends this to a self-referential evolutionary process in which both the task prompts and the mutation operators are improved jointly. These approaches demonstrate that prompts are not fixed assets but can be optimised against a measurable reward signal—essentially applying the reinforcement-learning-from-feedback idea at the prompt level rather than the weight level.

Per-item context as an optimisable unit. The Ad LLM platform applies this principle at the granularity of individual ads. Each ad in the inventory carries, in addition to its creative text and targeting metadata, a free-text “presentation context guide” that is injected into the response-synthesis prompt whenever that ad is matched. This guide specifies how the product should be framed: tone, recommended angles, features to emphasise, situations in which to avoid mentioning it, and comparisons to avoid. It is distinct from the creative text in that it constrains the *model’s rendering* of the product rather than the copy shown to the user. The context guide is versioned: each update is persisted alongside the engagement metrics snapshot that justified it, yielding an auditable history of how the framing of each ad has evolved. The initial version of each context guide is itself generated automatically when the ad is first created: a bootstrap LLM call inspects the ad’s creative text, targeting metadata, and (where available) scraped product-page content, and drafts a first-pass guide, so that the optimisation loop begins from a reasonable starting point rather than from an empty string.

Closing the loop with engagement feedback. The multi-dimensional engagement vector introduced above serves as the reward signal for an automatic optimiser. At a configurable cadence, and subject to minimum-sample and cooldown guards, a separate LLM call reviews the recent engagement history for an ad—aggregate engagement rate, average engagement score, average naturalness score, average purchase proximity, positive and negative sentiment counts, dismissal counts, and a sample of raw follow-up messages—together with the current context guide, and proposes a revised guide with an accompanying natural-language rationale. The optimiser prompt encodes diagnostic heuristics such as "if naturalness is low, soften the tone" and "if dismissals are high, tighten the relevance criteria." The resulting system implements a closed loop analogous in spirit to bandit-style ad optimisation [26], but operating over natural-language prompt fragments rather than numeric bid or placement parameters, and using an LLM both to interpret the feedback and to author the next candidate. To the best of our knowledge, no prior published work applies per-item LLM-driven prompt optimisation to a conversational advertising setting.

Relationship to reinforcement learning from human feedback. RLHF [32] trains model weights against a reward model learned from human preference data. The adaptive context mechanism adopts the same underlying intuition—behaviour should be shaped by downstream feedback—but differs in two important ways. First, the feedback signal is produced by an LLM judge rather than human raters, following the LLM-as-judge paradigm discussed above. Second, no gradient updates are applied to the base model; adaptation happens entirely in the text of the prompt context, which is cheap, reversible, and interpretable. This positions the approach closer to *prompt-level learning* than to parameter-level fine-tuning, and makes it attractive for deployment scenarios in which the base model is a third-party API whose weights are inaccessible.

2.8 Generative Image Models and Native Product Placement

Text-to-image and image-editing models. Recent text-to-image systems—DALL-E 3 [5], Stable Diffusion [34], and Imagen [36]—produce photorealistic images from natural-language prompts and have become standard components of generative chat interfaces. A parallel line of work on image editing and subject-driven generation, including DreamBooth [35] and InstructPix2Pix [6], enables a specific visual subject to be inserted or preserved across generations by conditioning on one or more reference images in addition to the text prompt. These capabilities are now exposed as image-editing endpoints in commercial APIs, allowing a reference image to act as a visual anchor for the generated scene.

Product placement as a creative convention. In traditional media—film, television, and more recently influencer video—product placement is a long-established advertising format in which a branded item is integrated diegetically into the scene rather than presented as an interruption [4]. Empirical studies have shown that well-executed placement can achieve higher recall and more favourable brand attitudes than conventional spot advertising, provided the placement feels natural to the narrative context [33]. Until recently, this format required human creative direction and physical staging; generative image models now make algorithmic placement technically feasible at per-request granularity.

Extending the Ad LLM thesis to the visual modality. When a user issues an image-generation request through the Ad LLM platform, the same context-extraction and RAG-based ad-matching pipeline that handles chat messages is invoked to select a relevant product. An auxiliary LLM call then rewrites the user's original image prompt so that the matched product appears organically in the scene (e.g., "on a shelf", "held by a subject", "worn by a runner"), preserving the user's original creative intent while adding the product as a diegetic element. If the ad inventory includes a reference photograph of the product, the rewritten prompt is passed to an image-editing endpoint together with the reference image, so that the generated scene is visually grounded in the actual product rather than in the model's prior over product names. This directly mirrors the text pipeline's "instruct the generative model to weave in the ad" design philosophy and extends it to pixels, producing what is, to our knowledge, the first described pipeline for *RAG-matched, sponsored product placement in user-requested AI-generated imagery*.

Open questions specific to the visual modality. Placing real branded products in generated images raises ethical and legal questions that do not arise in the text case. Visual disclosure is harder than textual disclosure: a `[Sponsored]` tag can be prepended to a response caption, but the image itself carries no intrinsic marker that the product within it is a paid placement. Intellectual-property considerations are also sharper, since a generated image of a branded product may implicate trademark and trade-dress rights that a textual mention does not. These issues are acknowledged as open and are discussed further in the ethical considerations of Section **??**; the literature on regulatory responses to AI-generated visual content is still nascent and, at the time of writing, does not yet provide clear guidance on sponsored generative imagery.

2.9 Summary and Research Gap

The preceding subsections have surveyed eight bodies of work that converge on the problem addressed by this project: (i) the evolution of digital advertising toward native, content-integrated formats; (ii) the capabilities of transformer-based LLMs for instruction following, structured output, and tone-adapted generation; (iii) the RAG paradigm for grounding language-model output in retrieved external context; (iv) the emerging practice of embedding advertising in conversational AI interfaces, together with the ethical and regulatory constraints that govern disclosure; (v) prompt engineering techniques for controlled generation and the LLM-as-judge paradigm for both offline and in-the-loop quality evaluation; (vi) engagement metrics that go beyond click-through to capture genuine user attention in dialogue systems, decomposed into a multi-dimensional signal vector; (vii) automatic and data-driven prompt optimisation as a lightweight alternative to parameter-level fine-tuning; and (viii) generative image models and the long-standing advertising convention of product placement, which together make algorithmic visual ad integration newly feasible.

The gap that this project addresses lies at the intersection of these strands. Existing industry deployments of conversational advertising—Microsoft's Bing Chat ads and Google's SGE/AI Mode placements—insert conventional ad units (text links, product cards) alongside AI-generated responses rather than instructing the language model to integrate product mentions into the response itself. Academic work on native advertising has extensively studied user perception and disclosure effectiveness, but has not examined LLM-generated native advertising in which the model itself produces the sponsored content. The RAG literature provides the retrieval infrastructure for context-grounded generation, but has not applied the retrieve-then-generate pipeline to ad matching and integration. Prompt engineering research offers the techniques for controlled output, but has not specifically addressed the dual-objective problem of simultaneous helpfulness and natural ad integration.

No open, well-documented architecture currently exists that combines semantic ad matching via a specialised RAG pipeline, LLM-driven natural ad integration with mandatory disclosure, sensitivity-aware ad gating, and engagement-based measurement within a single end-to-end system. The *Ad LLM as a Service* platform is designed to address this gap: it provides a complete, working pipeline from user message to ad-augmented response, accompanied by an analytics framework that measures conversational engagement as a first-class advertising metric.
===

3 Problem Statement

4 Problem Statement

This section formalises the problem addressed by the present work. We first explain why the inventory of advertising formats inherited from the visual web fails to transfer to conversational interfaces, then enumerate the technical sub-problems that any LLM-native advertising system must solve, then state the ethical constraints that bound the admissible solution space, and finally present a precise formal definition of the task.

4.1 Limitations of Traditional Ad Formats in Conversational AI

Three families of advertising formats dominate the contemporary web, and each fails to transfer to a conversational setting for a structural reason rather than an incidental one.

Display advertising—banners, sidebars, and in-feed units—presupposes a two-dimensional visual canvas in which a rectangular region can be reserved for sponsored content without disturbing the primary content stream. A chat interface offers no such region. The output of an LLM is a single linear sequence of tokens rendered as flowing text, and any attempt to reintroduce a sidebar or banner reasserts the very visual paradigm that conversational systems were designed to replace.

Interruption-based formats—pre-roll video, interstitials, and modal overlays—monetise attention by suspending the user's task for a fixed duration. In a conversational exchange, an interruption between the user's question and the assistant's answer breaks the implicit contract that the system responds to what was asked. The cost of such an interruption is not merely an aesthetic complaint; it directly degrades the property—fluid turn-taking—that makes conversational interfaces valuable in the first place.

Search advertising appears at first glance to be the closest analogue, since both search and chat are text-driven and intent-rich. The analogy is nevertheless misleading. A search query is a short, atomic keyword expression matched against a bid space by exact and approximate string operators. A conversational query is multi-turn: the relevant intent is distributed across several user messages, refined by clarifying questions, and frequently never expressed as a single noun phrase. A keyword auction operating on the most recent user message alone discards the bulk of the signal that the conversation contains.

The conclusion is that advertising in conversational AI is not a porting problem. It requires a format that is itself conversational—one in which sponsored content is integrated into the assistant's reply as a natural recommendation rather than appended as a foreign object.

4.2 Technical Challenges

Designing such a format raises four technical sub-problems that must be solved jointly.

4.2.1 Real-Time Intent Extraction from Multi-Turn Conversations

Let $H_t = (m_1, m_2, \ldots, m_{t-1})$ denote the conversation history up to turn t and m_t the current user message. The system must produce, in real time, a structured representation $C_t = \phi(H_t, m_t)$ that captures the user's latent commercial intent, the topical focus of the conversation, the strength of any purchase signal, and the user's receptivity to advertising at the present moment. The mapping ϕ cannot be reduced to keyword extraction: purchase intent often emerges only when a sequence of exploratory questions is considered jointly, and receptivity depends on conversational register (casual chit-chat, technical research, emotional disclosure) rather than on any single lexical cue. The extraction must additionally complete within a latency budget compatible with interactive use, ruling out heavyweight inference.

4.2.2 Semantic Ad Matching at Low Latency

Given the extracted context C_t and an ad inventory $\mathcal{A} = \{a_1, \ldots, a_N\}$, the system must select a small set of candidate advertisements that are semantically aligned with the conversation. Lexical matching against ad metadata is insufficient: a user discussing "training for a half marathon" should surface running shoes even though the phrase "running shoes" never appears in the dialogue. The matching function must therefore operate in a semantic space, must respect business constraints (active campaigns, positive remaining budget, frequency caps), and must be computable within a few hundred milliseconds over inventories that may grow to tens of thousands of items. A composite scoring rule is required, in which semantic similarity, advertiser bid, and the inferred purchase signal are combined into a single ranking criterion whose weights are interpretable and auditable.

4.2.3 Natural Ad Integration Without Degrading User Experience

Even a perfectly matched advertisement is worthless if it is appended to the response as a foreign block. The selected ad $a^\star$ must be woven into the assistant's reply as a recommendation that flows from the surrounding text, while a mandatory disclosure marker—we adopt the literal token `[Sponsored]`—must accompany every commercial mention to satisfy regulatory requirements. Two failure modes must be excluded by construction: the assistant must not promote $a^\star$ when no candidate is genuinely relevant to the user's question, and it must not produce a response in which the disclosure is omitted, hidden, or rephrased. Both constraints place a non-trivial burden on the prompt that drives the response-synthesis stage, since the model must be simultaneously encouraged to recommend and permitted to refuse.

4.2.4 Engagement Measurement Beyond Clicks

Click-through rate, the canonical metric of digital advertising, is poorly suited to conversational interfaces. Clicks are sparse—most users do not leave the conversation to visit an external page—and a click measures only the act of departure, not the quality of attention. A more faithful signal is available in the conversation itself: when a user follows a sponsored mention with a question about the advertised product, that follow-up is direct evidence that the advertisement entered the user's working model of the exchange. Detecting such follow-ups requires a classifier that takes both the prior advertisement and the next user message as input, and that distinguishes genuine engagement from coincidental topical overlap. We treat this engagement event as a first-class metric, complementary to and stronger than the click.

4.3 Ethical and Regulatory Considerations

Conversational advertising operates in a setting of elevated user trust. A chat assistant is perceived—accurately or not—as a neutral helper, and any recommendation it issues carries the weight of that perceived neutrality. This raises three constraints that the system must respect.

Disclosure. United States Federal Trade Commission guidance requires that sponsored content be identified clearly and conspicuously. We adopt the explicit token `[Sponsored]` adjacent to every commercial mention, and treat the absence of this token as a hard failure of the response-synthesis stage rather than a stylistic preference.

Sensitivity gating. Certain conversational contexts—disclosures of illness, bereavement, financial distress, or acute emotional difficulty—are incompatible with advertising of any kind. The context-extraction stage must therefore emit not only a positive receptivity score but also an explicit *none* value that causes the matching pipeline to be bypassed entirely. This is treated as an inviolable safeguard: under sensitivity gating, no advertisement is shown regardless of bid, similarity, or budget.

Data minimisation. Session state is retained only for the duration of an active conversation, with a fixed inactivity expiry. No persistent user profile is constructed across sessions, and no session content is shared with advertisers beyond the aggregate impression and engagement counters required for billing.

These constraints are not soft preferences. They define the admissible region of the design space and any solution that violates them is rejected *a priori*, irrespective of its performance on other metrics.

4.4 Formal Problem Definition

We may now state the problem precisely. Let $\mathcal{M}$ denote the space of natural-language messages and $\mathcal{A}$ a finite ad inventory. At turn t the system observes a history $H_t \in \mathcal{M}^{t-1}$ and a user message $m_t \in \mathcal{M}$, and must produce a tuple

$$(r_t,\ \alpha_t,\ e_t)\ \in\ \mathcal{M} \times (\mathcal{A} \cup \{\varnothing\}) \times \mathcal{E},$$

where r_t is the assistant's reply, α_t is the advertisement embedded in r_t (or $\varnothing$ if none is shown), and e_t is the set of engagement events logged for downstream analytics. The tuple is required to satisfy the following conditions.

(C1) Helpfulness.
The reply r_t answers the user's query in m_t on its own merits, independently of whether an advertisement is included.

(C2) Relevance-conditional injection.
If $\alpha_t \neq \varnothing$, then α_t is semantically aligned with the inferred context $C_t = \phi(H_t, m_t)$ and lies within the active, budgeted, and frequency-cap-eligible subset of $\mathcal{A}$.

(C3) Mandatory disclosure.
If $\alpha_t \neq \varnothing$, then r_t contains the disclosure marker `[Sponsored]` adjacent to the mention of α_t.

(C4) Sensitivity gating.
If C_t indicates a sensitive context, then $\alpha_t = \varnothing$ and no element of $\mathcal{A}$ is evaluated for inclusion.

(C5) Latency.
The end-to-end production of (r_t, α_t, e_t) completes within a fixed wall-clock budget compatible with interactive chat.

The objective is to design a system that satisfies (C1)–(C5) on every turn while maximising a composite utility that rewards both user-perceived helpfulness and verifiable advertiser engagement. The remainder of this report develops such a system, presents the architectural decisions that realise each of (C1)–(C5), and evaluates the resulting platform on a curated set of representative conversations.

5 Solution Design

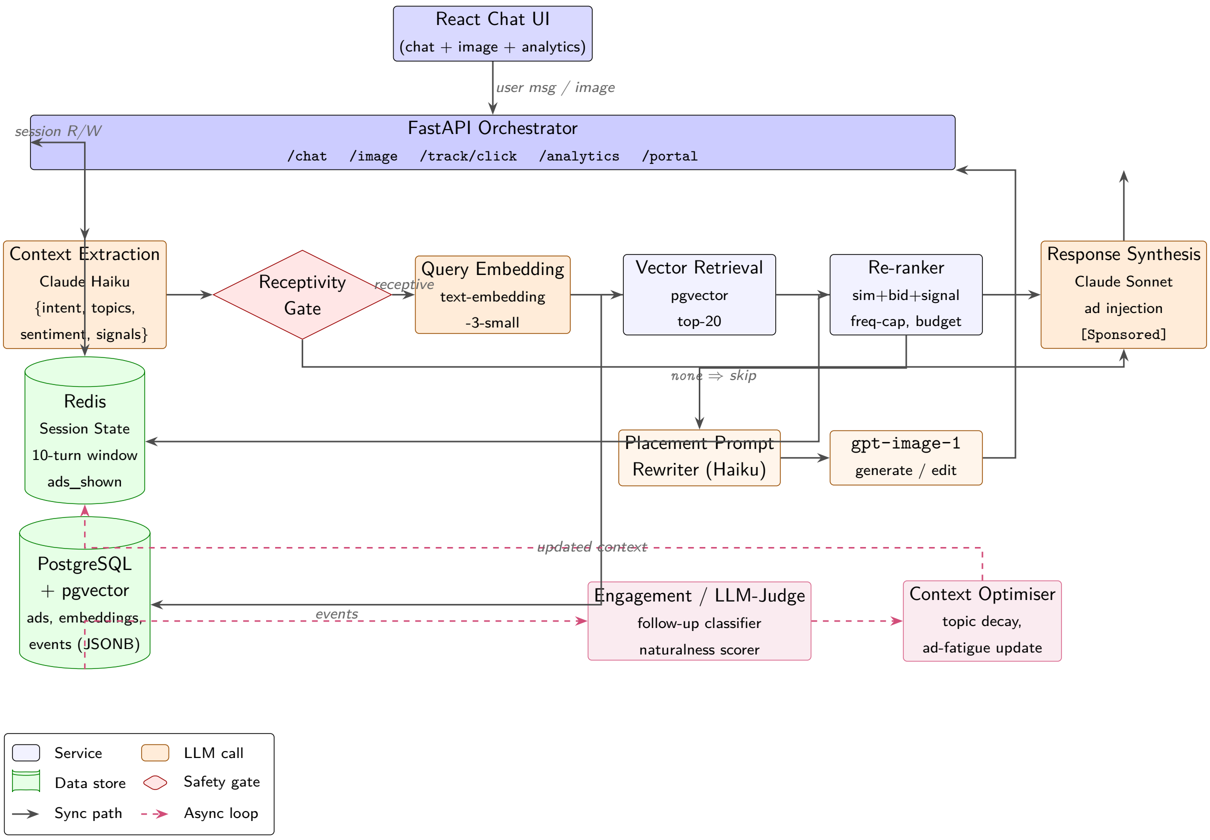

Figure 1. High-level system architecture of the Ad LLM platform...

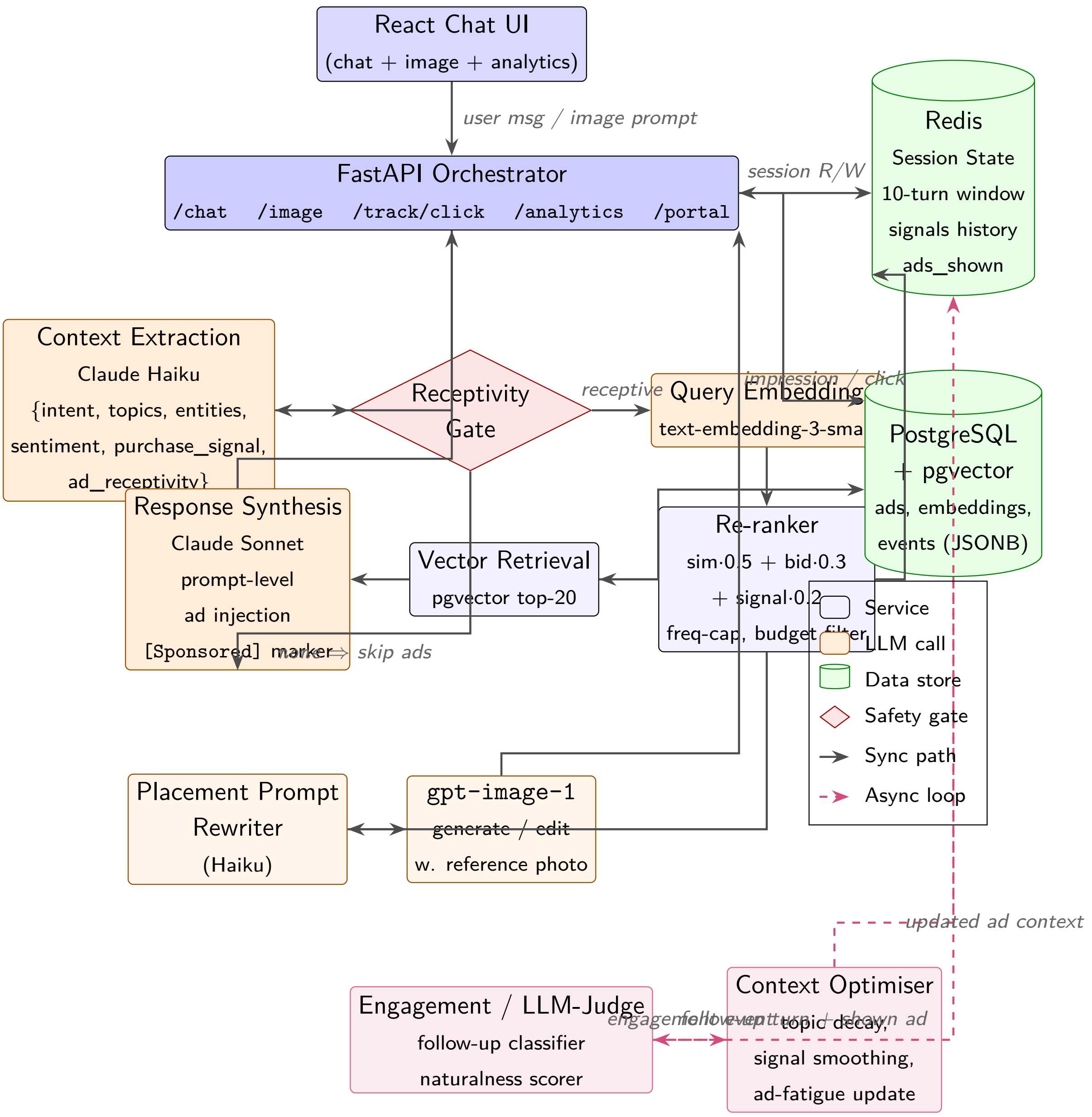

Figure 2. High-level system architecture of the Ad LLM platform.

Table 1. Technology stack summary.

Component	Technology
Frontend	React
Orchestrator API	FastAPI (Python)
Session Store	Redis
Database	PostgreSQL + pgvector
Embedding Model	text-embedding-3-small (OpenAI)
Context Extraction LLM	Claude Haiku (lightweight)
Response Generation LLM	Claude Sonnet (capable)
Image Generation Model	`gpt-image-1`
Event Logging	PostgreSQL (JSONB payload)

5.1 Request-Path Components

This subsection describes the synchronous, user-facing pipeline: the sequence of components that a single user message—or a single image request—traverses between arrival at the orchestrator and the return of the rendered response. Asynchronous concerns such as event logging, analytics rollups, and the advertiser portal are deferred to Section **??**. The components below are presented in the order in which they execute on the request path, and each is characterised by its inputs, its outputs, and the failure mode it adopts when a downstream dependency is unavailable.

5.1.1 Orchestrator API (FastAPI)

The orchestrator is a single FastAPI service that exposes the public HTTP surface of the platform and coordinates every internal module involved in producing a response. It is deliberately monolithic: context extraction, ad retrieval, response synthesis, session management, and event logging are mounted as in-process modules behind a thin router layer rather than as independent services. This choice trades the horizontal scalability of a microservice topology for two properties that matter more at the MVP scale, namely a single deployment artefact and the elimination of inter-service network hops from the critical latency path.

The principal endpoint is `POST /chat`, whose handler implements a seven-stage pipeline. The handler first loads the conversation state for the supplied `session_id` from Redis and, if the previous turn surfaced an advertisement, dispatches an engagement-detection call against the user's new message before any other work proceeds. The user's turn is then persisted, and the message together with the rolling history is passed to the context extraction module. If the extracted `ad_receptivity` field is anything other than `none`, the handler issues an embedding call on the extracted context, retrieves a re-ranked candidate set from the ad service, generates per-impression tracking URLs, and invokes response synthesis with the chosen ad bound into the prompt. If receptivity is `none`—the safety branch—the same synthesis call is made with an empty ad list, so that sensitive conversations follow an identical code path but never carry sponsored content. The handler finally appends the assistant turn to Redis, logs an impression event if an ad was injected, and returns a response payload containing the rendered text and, when applicable, the ad metadata required by the frontend to render the `[Sponsored]` badge.

Two design decisions deserve emphasis. First, every call into the AI modules is wrapped in a guarded `try/except` block whose fallback is to continue the pipeline without the failed signal: a context-extraction failure degrades the request to an unsponsored response rather than to an error, and a response-synthesis failure is the only condition under which the endpoint returns a non-2xx status. The orchestrator therefore treats advertising as a strictly additive overlay on a working chat service, never as a precondition for it. Second, the secondary endpoints—`GET /track/click` for click attribution, `GET /analytics/summary` for the dashboard, and the `/portal/*` family for advertiser self-service—are mounted on the same application but execute outside the latency-critical chat path and are therefore free to perform synchronous database work that the chat handler would have to defer.

5.1.2 Context Extraction Module

The context extraction module is the first AI call on the request path and the component on which every subsequent ad-related decision depends. It receives the user's current message together with the rolling conversation history and returns a structured object whose fields are the intent label, a list of topics, a list of named entities, a coarse sentiment value, a continuous purchase signal in the closed unit interval, and a four-valued ad-receptivity label drawn from $\{\texttt{high}, \texttt{medium}, \texttt{low}, \texttt{none}\}$. This object is the sole interface between raw conversational text and the retrieval and synthesis stages that follow; no downstream module re-reads the user message for ad-related decisions.

The module is implemented as a single call to a lightweight chat model, chosen so that the extraction step contributes only marginally to the end-to-end latency budget while still benefiting from the in-context reasoning that distinguishes, for example, a product research query from a venting message that happens to mention a brand. The prompt instructs the model to return strict JSON conforming to the schema above, and the orchestrator validates the response against a Pydantic model before propagating it; a parse failure is treated as a soft error and degrades the request to the unsponsored branch.

The `ad_receptivity` field carries a load that the other fields do not. It is the platform's safety gate: when the extractor labels a message as describing a health crisis, an episode of emotional distress, or a financial hardship, the field is set to `none` and the orchestrator unconditionally skips ad retrieval and ad injection for that turn. This places the safety decision at the earliest possible point in the pipeline, before any embedding or retrieval work has been performed, and ensures that sensitive conversations cannot incur an impression even accidentally. The remaining three values are interpreted as a soft gate by the re-ranker rather than as hard switches, so that a borderline-receptive turn can still produce a recommendation when the semantic match to inventory is sufficiently strong.

5.1.3 Session Management (Redis)

Session state is held in a single Redis instance and keyed by the `session_id` that the frontend generates at the start of each conversation. Each session record contains a rolling window of the ten most recent turns, an accumulated list of topics observed across the conversation, the history of purchase signals reported by the context extractor, the identifiers of advertisements already shown in the session, and a turn counter. The rolling window bounds both the memory footprint per session and the number of tokens that the context extractor and the response synthesiser must process on each turn, which is the dominant cost driver in both calls.

Three properties of this design are worth noting. First, the history of purchase signals enables the re-ranker to detect monotone escalation across turns, which is a stronger purchase indicator than any single message in isolation; the platform exploits this without requiring the LLMs themselves to reason over multi-turn dynamics. Second, the set of advertisements already shown is consulted by the re-ranker to suppress immediate repetition, which would otherwise be the dominant failure mode of a purely similarity-driven retriever in a long conversation about a single topic. Third, because Redis is the only stateful dependency on the chat path that the orchestrator writes to synchronously, the platform can tolerate a Redis outage by falling back to a stateless mode in which every turn is treated as the first turn of a fresh conversation; the user observes a loss of personalisation but not a loss of service.

5.1.4 Response Synthesis with Ad Injection

Response synthesis is the second and more expensive of the two LLM calls on the request path. It receives the user's message, the rolling conversation history, the structured context object produced by the extractor, the chosen advertisement (if any), and the pre-generated tracking URL associated with that impression. Its output is a single markdown-formatted assistant message which the frontend renders directly into the chat transcript.

The defining design decision is that ad injection is realised as *prompt-level conditioning* rather than as post-hoc text rewriting. When an advertisement is selected, the synthesiser's system prompt is augmented with a sponsored block that contains the product name, its description, the advertiser-supplied creative copy, any campaign-level presentation guidance, and a strict instruction to weave the product into the response as a markdown link of the form `[Sponsored] [product](tracking_url)`, followed by a single compelling reason. The synthesis model is then free to determine where in its response the sponsored mention is introduced, which substantive parts of the user's question to address first, and how to phrase the recommendation so that it follows naturally from the surrounding answer. This delegation is what distinguishes the platform from a banner-style overlay: the model is not asked to attach an advertisement to a response, it is asked to write a response in which the advertisement participates.

When the receptivity gate has fired and no advertisement is supplied, the same synthesis call is made with an unmodified system prompt. The two branches share a single code path and a single model invocation, which guarantees that the unsponsored response distribution is not distorted by structural differences between the two branches and that a regression in the sponsored branch cannot silently corrupt the unsponsored one. The `[Sponsored]` marker and the tracking URL are the only artefacts by which a downstream observer—the user, the analytics dashboard, or a future audit—can distinguish a sponsored response from an organic one.

5.1.5 Image Generation with Product Placement

The image generation endpoint is the platform's second generative surface and the visual analogue of the chat pipeline described above. It is exposed as a distinct route on the same orchestrator and operates under a deliberately looser latency budget, since diffusion-model inference routinely exceeds ten seconds and is dominated by GPU time rather than by the orchestration overhead that constrains the text path. Its purpose is to demonstrate that the context-extraction and retrieval-augmented matching pipeline generalises beyond text and can deliver organic, RAG-matched product placement directly into user-requested imagery.

When a user submits an image prompt, the orchestrator routes the prompt through the same context extractor used on the chat path, issues an embedding call against the resulting context object, and retrieves a re-ranked candidate set from the ad service. If a sponsored product is selected, the original prompt is not passed to the image model directly. Instead, an auxiliary lightweight LLM call rewrites the prompt into a *placement prompt*: a new specification that preserves the user's stated subject, style, and composition while introducing the matched product as a diegetic element of the scene—worn by a subject, held by a hand, resting on a surface, displayed on a shelf—chosen to fit the semantics of the original request rather than to dominate it. The rewritten prompt is then passed to the image model, `gpt-image-1`, which renders the final image.

A second branch of the pipeline activates when the matched advertisement carries a reference photograph of the product in the ad inventory. In this case the rewritten prompt is dispatched to the model's image-edit endpoint together with the reference image, so that the rendered product is visually grounded in the actual article rather than reconstructed from the model's prior over the product name. This is a small change in the API call but a significant change in the semantics of the output: it is what allows a generated scene to depict a recognisable, brand-accurate product rather than a plausible-looking generic substitute, and it is what makes the visual pipeline a meaningful extension of the textual "instruct the generative model to weave in the ad" design philosophy rather than a cosmetic copy of it. As with the chat path, a failure anywhere in the ad-related branches degrades the request to an unsponsored image generation: the user always receives an image, and the sponsored overlay is strictly additive.

5.2 Ad Serving and Matching

This subsection describes the path a user turn takes from the moment the orchestrator decides that an advertisement may be appropriate, to the moment a small ranked set of candidate ads is handed back to the response synthesis stage. We treat the problem as a two-stage retrieval-and-ranking pipeline: a high-recall semantic retrieval over a vector index, followed by a business-rules re-ranking step that enforces the economic and policy constraints of the platform. This decomposition mirrors the structure of modern web search and recommendation systems, where embedding-based recall is combined with a lightweight learned or hand-tuned re-ranker **karpukhin2020dense**, but is adapted here to the constraints of a multi-turn conversational setting.

5.2.1 Ad Inventory, Campaigns, and Advertisers (PostgreSQL + pgvector)

The ad inventory is the system of record for every entity an advertiser cares about: the advertiser account itself, the campaigns it funds, and the individual creatives that may be served. We store all three in a single PostgreSQL instance, extended with the `pgvector` extension to support approximate nearest-neighbour search over dense embeddings. The decision to co-locate relational and vector data in PostgreSQL, rather than to introduce a dedicated vector database such as Pinecone or Weaviate, is deliberate. At the scale we target in this report (on the order of 10^3–10^5 ads), `pgvector` with an IVFFlat index comfortably meets our latency budget, and keeping ads, budgets, and embeddings in the same transactional store removes an entire class of consistency bugs that arise when budget decrements and embedding lookups live in different systems.

The schema is organised around three tables. The `advertisers` table holds account-level metadata and contact information. The `campaigns` table groups ads under a shared budget envelope, bid strategy, and flight window, and references an advertiser via a foreign key. The `ads` table holds the per-creative fields that the matching pipeline actually reads at request time:

- **Identity and content:** `ad_id`, `title`, `description`, `cta_url`, `product_category`.
- **Targeting:** a free-text `targeting_context` field that captures the kinds of conversational situations the advertiser wants to reach (e.g. “users researching marathon training shoes”).
- **Economics:** `bid_cpm`, `remaining_budget`, `daily_cap`, and `active` flag.
- **Embedding:** a `vector(1536)` column populated at ingestion time by the embedding pipeline of Section 5.2.2.

An IVFFlat index is built on the embedding column using cosine distance as the similarity operator. The choice of cosine over inner product is motivated by the fact that the OpenAI `text-embedding-3-small` embeddings used in this work are not unit-normalised in a way that makes raw inner products meaningful for ranking across heterogeneous ad copy lengths. Cosine similarity is invariant to embedding magnitude and therefore yields more stable rankings when ad descriptions vary in length, which is the common case in our inventory.

The `events` table, although logically part of the feedback loop described in Section 5.4, is co-located in the same database so that impression and click writes can participate in the same transaction as the budget decrement on the corresponding ad row. This is what allows us to make a strong claim later: a charged impression and a budget deduction either both happen or neither does.

5.2.2 Embedding and Semantic Matching Pipeline

Semantic matching in the system is built on the principle that the ad and the conversational context should be projected into the *same* vector space, but from *different* textual surfaces. On the ad side, we embed a concatenation of the title, description, product category, and targeting context; on the query side, we embed a structured serialisation of the output of the context extraction module (Section 5.1.2), not the raw user message.

This asymmetry is important and worth justifying. The raw user message is a poor query for two reasons. First, it is often short, casual, and lacks the domain vocabulary that ad copy is written in — a user who says “my knees are killing me after long runs” shares almost no surface tokens with an ad titled “Nike Pegasus 41 — Cushioned Daily Trainer”. Second, intent in a conversational setting is distributed across turns, so any embedding of a single message systematically discards the accumulated context that the session store has been maintaining. By embedding the structured context object instead — which contains the extracted intent, topics, entities, and purchase signal — we obtain a query representation that is both richer and more lexically aligned with the way advertisers describe their products.

Formally, let $f_\theta : \Sigma^* \to \mathbb{R}^{1536}$ denote the embedding model. For an ad a with textual fields $T(a)$, the ingestion-time embedding is $\mathbf{e}_a = f_\theta(T(a))$, computed once and persisted in the `ads` table. For a user turn m_t with conversation history H, let $C = \texttt{extract\_context}(m_t, H)$ be the structured context, and let $S(C)$ be its canonical serialisation. The query embedding is then $\mathbf{q} = f_\theta(S(C))$, computed at request time. Retrieval reduces to finding the top-N ads under cosine similarity:

$$\mathit{Retrieve}(\mathbf{q}, N) \; = \; \arg\operatorname*{top-}N_{a \in \mathcal{A}} \; \frac{\mathbf{q}^\top \mathbf{e}_a}{\|\mathbf{q}\| \, \|\mathbf{e}_a\|}.$$

We set $N = 20$, which empirically gives the downstream re-ranker enough diversity to enforce budget and frequency constraints without exhausting the inventory in low-fill regimes.

Two practical optimisations are worth noting. First, the embedding call for the query is the second most expensive synchronous step in the pipeline (after response generation), so the orchestrator issues it in parallel with the tail end of context extraction whenever the structured context becomes incrementally available. Second, the IVFFlat index is warmed at process start by issuing a dummy nearest-neighbour query, which avoids a cold-cache penalty on the first real user turn after a deployment.

5.2.3 Re-Ranking, Budget Gating, and Business Logic

The output of semantic retrieval is a high-recall set of $N = 20$ candidates that are *topically* plausible but not yet economically or contextually admissible. The re-ranker is the component that turns this candidate set into a small ranked list ready for the response synthesis stage. It performs three jobs in sequence: *filtering*, *scoring*, and *selection*.

Filtering. A candidate ad a is admissible only if it satisfies the conjunction of the following predicates: (i) a.`active` is true, (ii) a.`remaining_budget` $> a$.`bid_cpm`$/1000$ (i.e. the campaign can afford one more impression), and (iii) a.`ad_id` does not appear in the `ads_shown_this_session` list maintained by Redis. The third predicate is the per-session frequency cap and is the simplest but most user-visible safeguard against repetition fatigue: a user who has already seen a Nike Pegasus mention in turn 3 will not see it again in turn 5, even if it remains the highest-similarity candidate. Filtering is performed in application code rather than as part of the SQL query because the session-level shown set lives in Redis, not in PostgreSQL, and we prefer to keep that boundary explicit.

Scoring. For each surviving candidate we compute a composite score that blends semantic relevance with the platform's economic objective and the inferred user state. Let $\mathrm{sim}(a)$ denote the cosine similarity between $\mathbf{q}$ and $\mathbf{e}_a$, and let $b_{\max} = \max_{a' \in \mathit{candidates}} a'$.`bid_cpm` be the maximum bid in the surviving set. We define

$$\mathit{score}(a) \; = \; 0.5 \cdot \mathrm{sim}(a) \; + \; 0.3 \cdot \frac{a.\texttt{bid\_cpm}}{b_{\max}} \; + \; 0.2 \cdot \mathit{purchase\_signal}(C),$$

where $\mathit{purchase\_signal}(C) \in [0, 1]$ is the scalar field produced by the context extraction module. Normalising bid by $b_{\max}$ rather than by a global constant keeps the bid term on the same scale as the other two across very different inventories, which is important during early-stage testing when the inventory is small and bid magnitudes are arbitrary.

The weights $(0.5, 0.3, 0.2)$ are deliberately chosen to make relevance the dominant factor: even the highest bidder cannot displace a much more relevant ad, and a high purchase signal can act as a tie-breaker between two ads of similar relevance. We treat these weights as hand-tuned hyperparameters in this report; learning them from logged engagement data is a natural extension and is discussed in Section 7.4.

Selection. The top $k = 3$ scored candidates are returned to the orchestrator. Returning more than one allows the response synthesis prompt (Section 5.1.4) to gracefully fall back to a second-choice ad if the LLM judges the top candidate to be a poor contextual fit, without requiring a second round-trip to the database.

Algorithm **??** summarises the complete procedure as implemented in `get_candidate_ads`, which is the function exposed to the AI workstream through the interface contract of Table **??**.

5.3 Feedback Loop and Adaptation

The components described so far are sufficient to serve a single ad on a single turn, but they are not sufficient to make the system *learn* from its own behaviour. This subsection describes the asynchronous feedback loop that observes user reactions to served ads, distinguishes genuine engagement from noise, and feeds that signal back into the matching pipeline. We regard this loop as one of the central contributions of the project, because it is the part of the design that takes seriously the observation that conversational ads cannot be evaluated by clicks alone.

5.3.1 Click Tracking, Event Logging, and Budget Accounting

Every ad that appears in a user-facing response carries a call-to-action URL. Rather than expose the advertiser's destination URL directly, the orchestrator rewrites it at synthesis time into a tracked redirect of the form `/track/click?ad_id=X&session_id=Y&redirect=Z`, where Z is the URL-encoded original destination. When the user clicks the link, the `/track/click` endpoint writes a `click` event to the `events` table and issues a 302 redirect to Z. The user perceives a single click; the system gains a logged interaction tied to the originating session and ad.

The event model is uniform across the four event types we record: `impression`, `click`, `engagement`, and `conversion`. Each row carries an `event_id`, an `ad_id`, a `session_id`, a `turn_index`, a UTC timestamp, and a JSONB `payload` column for type-specific fields (e.g. the classifier confidence for an engagement event, or the referring turn for a click). Storing the type-specific fields in JSONB rather than in separate tables keeps the analytics queries simple — a single `GROUP BY event_type` suffices to compute most dashboard metrics — at the cost of giving up some compile-time schema enforcement, which we judge an acceptable trade for a research prototype.

Budget accounting is tied to the impression event. When the orchestrator logs an impression, it does so in the same PostgreSQL transaction that decrements `remaining_budget` on the corresponding ad row by `bid_cpm`/1000. Because the event insert and the budget decrement are atomic, the system maintains the invariant that the sum of charged impressions per ad, multiplied by its CPM, equals the total amount drawn down from its budget. This is the property that lets the analytics layer report revenue numbers that reconcile with the database rather than drift from it over time.

5.3.2 LLM-Judge-Based Engagement Detection

Clicks are a poor primary signal in a conversational setting. A user who is intrigued by a sponsored mention is far more likely to ask a follow-up question about the product than to click an inline link, and yet that follow-up is exactly the kind of behaviour that classical ad analytics is blind to. The engagement detection module is the system's answer to this mismatch: it inspects the user's *next* turn after an ad was served and decides whether that turn constitutes engagement with the advertised product.

We implemented this in two stages. The first stage is a cheap keyword and entity matcher: if the next user message contains the ad's product name, brand, or any of the entities extracted from its title, we tentatively flag the turn as engaged. This catches the easy cases ("tell me more about the Pegasus 41") at essentially zero cost. The second stage, invoked only when the first stage is ambiguous, is a lightweight LLM judge — the same Haiku-tier model used for context extraction — prompted with the served ad, the previous assistant turn, and the new user turn, and asked to return a boolean `engaged` field together with a short rationale and a confidence score in $[0, 1]$.

The two-stage design is a deliberate latency and cost optimisation. The keyword stage handles the majority of true positives without any model call; the LLM stage handles the genuinely ambiguous cases, where the user paraphrases ("how durable are those shoes you mentioned?") or asks an implicit follow-up. When the LLM judge returns `engaged = true` with confidence above a threshold $\tau = 0.7$, an `engagement` event is written to the events table with the rationale stored in the JSONB payload. We treat this event as a strictly stronger signal than a click, both in the analytics dashboard and, as described next, in the adaptive layer that modulates future ad selection.

It is worth being explicit about what the engagement detector is *not*. It is not a sentiment classifier, and it does not try to decide whether the user liked the ad. It only decides whether the user's next turn is *about* the advertised product. Whether that engagement is positive, negative, or neutral is left to downstream analysis; what matters for the feedback loop is that the user's attention measurably moved towards the ad.

5.3.3 Adaptive Ad Context Optimisation

The final piece of the feedback loop closes it: engagement events are not merely logged for reporting, they actively reshape what the matching pipeline does on subsequent turns. We call this *adaptive ad context optimisation*, and it operates at two granularities.

Within-session adaptation. When an engagement event is recorded for an ad a in session s, the orchestrator updates the Redis session state for s in two ways. First, the topics and entities associated with a are merged into the session's `accumulated_topics` list, which biases the next round of context extraction towards the same product neighbourhood. Second, the session's `purchase_signal` is bumped upward by a fixed increment $\Delta = 0.15$, capped at 1.0. The effect on the very next turn is concrete and measurable: the query embedding shifts toward the engaged-with topic, and the re-ranker's purchase-signal term increases, jointly making it more likely that a related and complementary ad surfaces if one exists in the inventory.

Cross-session adaptation. At the inventory level, engagement and click events feed a slow-moving per-ad quality score $q_a \in [0, 1]$, computed as a smoothed engagement-plus-click rate over the ad's recent impressions. This score is not part of the per-request scoring formula in Section 5.2.3 — introducing it there would risk a runaway feedback loop in which a few early lucky impressions monopolise future serving — but it is exposed to the analytics dashboard and used to surface under-performing ads to the (currently manual) inventory review process. A learned re-ranker that incorporates q_a directly, with appropriate exploration, is the natural next step and is sketched in Section 7.4.

Taken together, these two granularities give the system a feedback loop with two distinct time constants: a fast, within-session loop that adapts the next turn's ad selection to what the user just engaged with, and a slow, cross-session loop that lets the inventory as a whole drift toward ads that consistently earn user attention. Neither loop requires labelled training data or a separate model training pipeline; both are driven entirely by the events that the system already records as part of its normal operation.

5.4 Feedback Loop and Adaptation

The components described so far are sufficient to serve a single ad on a single turn, but they are not sufficient to make the system *learn* from its own behaviour. This subsection describes the asynchronous feedback loop that observes user reactions to served ads, distinguishes genuine engagement from noise, and feeds that signal back into the matching pipeline. We regard this loop as one of the central contributions of the project, because it is the part of the design that takes seriously the observation that conversational ads cannot be evaluated by clicks alone.

5.4.1 Click Tracking, Event Logging, and Budget Accounting

Every ad that appears in a user-facing response carries a call-to-action URL. Rather than expose the advertiser's destination URL directly, the orchestrator rewrites it at synthesis time into a tracked redirect of the form `/track/click?ad_id=X&session_id=Y&redirect=Z`, where Z is the URL-encoded original destination. When the user clicks the link, the `/track/click` endpoint writes a `click` event to the `events` table and issues a 302 redirect to Z. The user perceives a single click; the system gains a logged interaction tied to the originating session and ad.

The event model is uniform across the four event types we record: `impression`, `click`, `engagement`, and `conversion`. Each row carries an `event_id`, an `ad_id`, a `session_id`, a `turn_index`, a UTC timestamp, and a JSONB `payload` column for type-specific fields (e.g. the classifier confidence for an engagement event, or the referring turn for a click). Storing the type-specific fields in JSONB rather than in separate tables keeps the analytics queries simple — a single `GROUP BY event_type` suffices to compute most dashboard metrics — at the cost of giving up some compile-time schema enforcement, which we judge an acceptable trade for a research prototype.

Budget accounting is tied to the impression event. When the orchestrator logs an impression, it does so in the same PostgreSQL transaction that decrements `remaining_budget` on the corresponding ad row by `bid_cpm`/1000. Because the event insert and the budget decrement are atomic, the system maintains the invariant that the sum of charged impressions per ad, multiplied by its CPM, equals the total amount drawn down from its budget. This is the property that lets the analytics layer report revenue numbers that reconcile with the database rather than drift from it over time.

5.4.2 LLM-Judge-Based Engagement Detection

Clicks are a poor primary signal in a conversational setting. A user who is intrigued by a sponsored mention is far more likely to ask a follow-up question about the product than to click an inline link, and yet that follow-up is exactly the kind of behaviour that classical ad analytics is blind to. The engagement detection module is the system's answer to this mismatch: it inspects the user's *next* turn after an ad was served and decides whether that turn constitutes engagement with the advertised product.

We implemented this in two stages. The first stage is a cheap keyword and entity matcher: if the next user message contains the ad's product name, brand, or any of the entities extracted from its title, we tentatively flag the turn as engaged. This catches the easy cases ("tell me more about the Pegasus 41") at essentially zero cost. The second stage, invoked only when the first stage is ambiguous, is a lightweight LLM judge — the same Haiku-tier model used for context extraction — prompted with the served ad, the previous assistant turn, and the new user turn, and asked to return a boolean `engaged` field together with a short rationale and a confidence score in $[0, 1]$.

The two-stage design is a deliberate latency and cost optimisation. The keyword stage handles the majority of true positives without any model call; the LLM stage handles the genuinely ambiguous cases, where the user paraphrases ("how durable are those shoes you mentioned?") or asks an implicit follow-up. When the LLM judge returns `engaged = true` with confidence above a threshold $\tau = 0.7$, an `engagement` event is written to the events table with the rationale stored in the JSONB payload. We treat this event as a strictly stronger signal than a click, both in the analytics dashboard and, as described next, in the adaptive layer that modulates future ad selection.

It is worth being explicit about what the engagement detector is *not*. It is not a sentiment classifier, and it does not try to decide whether the user liked the ad. It only decides whether the user's next turn is *about* the advertised product. Whether that engagement is positive, negative, or neutral is left to downstream analysis; what matters for the feedback loop is that the user's attention measurably moved towards the ad.

5.4.3 Adaptive Ad Context Optimisation

The final piece of the feedback loop closes it: engagement events are not merely logged for reporting, they actively reshape what the matching pipeline does on subsequent turns. We call this *adaptive ad context optimisation*, and it operates at two granularities.

Within-session adaptation. When an engagement event is recorded for an ad a in session s, the orchestrator updates the Redis session state for s in two ways. First, the topics and entities associated with a are merged into the session's `accumulated_topics` list, which biases the next round of context extraction towards the same product neighbourhood. Second, the session's `purchase_signal` is bumped upward by a fixed increment $\Delta = 0.15$, capped at 1.0. The effect on the very next turn is concrete and measurable: the query embedding shifts toward the engaged-with topic, and the re-ranker's purchase-signal term increases, jointly making it more likely that a related and complementary ad surfaces if one exists in the inventory.

Cross-session adaptation. At the inventory level, engagement and click events feed a slow-moving per-ad quality score $q_a \in [0, 1]$, computed as a smoothed engagement-plus-click rate over the ad's recent impressions. This score is not part of the per-request scoring formula in Section 5.2.3 — introducing it there would risk a runaway feedback loop in which a few early lucky impressions monopolise future serving — but it is exposed to the analytics dashboard and used to surface under-performing ads to the (currently manual) inventory review process. A learned re-ranker that incorporates q_a directly, with appropriate exploration, is the natural next step and is sketched in Section 7.4.

Taken together, these two granularities give the system a feedback loop with two distinct time constants: a fast, within-session loop that adapts the next turn's ad selection to what the user just engaged with, and a slow, cross-session loop that lets the inventory as a whole drift toward ads that consistently earn user attention. Neither loop requires labelled training data or a separate model training pipeline; both are driven entirely by the events that the system already records as part of its normal operation.

5.5 Advertiser-Facing Surfaces

The components described so far — matching, re-ranking, engagement detection, and the adaptive feedback loop — are all invisible to the advertiser. They live inside the request path of the chat service and communicate only with the assistant and the event store. For the system to function as a product rather than a research prototype, advertisers need a surface through which they can supply inventory, shape its targeting, fund its delivery, and inspect its performance. We divide this surface into two logically distinct layers: a transactional portal for managing campaign objects, and a read-only analytics layer for interpreting the events those campaigns generate.

5.5.1 Advertiser Portal and Campaign Management

The advertiser portal is a thin REST layer, mounted under the `/portal` prefix of the same FastAPI service that serves the chat endpoint, which exposes CRUD operations over three nested resources: *advertisers*, *campaigns*, and *ads*. An advertiser is the billing principal and owns one or more campaigns; a campaign groups ads under a shared budget and flight window; an ad is the smallest servable unit and carries the product metadata, targeting constraints, and creative text that the matching pipeline consumes.

The resource hierarchy is enforced in the URL structure — campaigns are reached as `/portal/advertisers/{adv` ads as `/portal/campaigns/{campaign_id}/ads` — so that ownership is unambiguous at every write and authorisation reduces to a prefix check. Each resource supports the usual five operations (`POST`, `GET` list, `GET` by id, `PATCH`, and soft delete via a status field), and every mutation is validated against a Pydantic schema before it reaches the database. Budgets are represented as two independent scalars, a total budget and a daily budget, both enforced in the delivery path by the pacing logic described in Section 5.2.3; a campaign whose daily spend has been exhausted is filtered out of the candidate set for the remainder of the day rather than being deleted or re-prioritised.

An important design choice is that the portal writes to the same PostgreSQL tables that the serving path reads from. There is no separate "staging" database and no publish step: a newly created ad becomes eligible for serving as soon as its row is committed and the embedding worker has populated its vector in pgvector. This keeps the write path simple and eliminates a class of consistency bugs in which portal state and serving state drift, at the cost of requiring the embedding worker to be online for new ads to become servable. We consider this an acceptable trade because the embedding worker is a small, stateless service whose failure is easy to detect and whose catch-up on restart is bounded by the number of ads created during its outage.

5.5.2 Analytics API and Dashboards

The analytics layer is intentionally a read-only projection over the events table rather than a second source of truth. Every metric the advertiser sees — impressions, clicks, engagements, click-through rate, engagement rate, estimated revenue — is computed by a SQL aggregation over the same `events` rows that the engagement detector writes in Section 5.4.2. This discipline is what allows the feedback loop and the reporting surface to remain consistent by construction: an engagement that reshapes the next turn's ad selection is the same row that increments the engagement count on the dashboard, and no reconciliation job is needed between them.

The API exposes three families of endpoints. A global summary endpoint, `GET /analytics/summary?days=N`, returns headline counters and a ranked list of top-performing ads over a rolling window and is used both by the internal admin view and by the top-of-dashboard cards on the advertiser side. A per-advertiser endpoint, `GET /portal/advertisers/{id}/analytics`, restricts the same aggregation to ads owned by a single advertiser and is the primary data source for the advertiser-facing dashboard. A per-campaign endpoint drills one level further and returns a daily time series suitable for the line charts on the campaign detail page. All three endpoints accept a `days` parameter and translate it into a SQL `WHERE timestamp >= NOW() - INTERVAL` clause; none of them maintain pre-aggregated tables, which we justify by noting that the events table is indexed on `(ad_id, event_type, timestamp)` and that the query volume from the dashboard is orders of magnitude below the write volume from the serving path.

The React dashboard that consumes these endpoints is deliberately unambitious in scope. It provides four pages — an overview with headline metrics and a time series chart, an advertisers list, a campaigns list scoped to a selected advertiser, and a campaign detail view with per-ad breakdowns — and uses Recharts for all visualisations. Its visual language is inherited directly from the chat interface's stylesheet so that the two surfaces read as a single product rather than two assemblages. The dashboard performs no client- side aggregation and no caching beyond React Query's default stale-while-revalidate behaviour; every number on the screen is a direct reflection of a server-side query over the events table at the time the page was loaded.

5.6 Interface Contracts and Workstream Division

A system of this size is only tractable to build if its seams are chosen deliberately rather than allowed to congeal around whoever happened to write the first line of code in each area. We therefore fixed the interfaces between major components in Week 1, before any production implementation existed on either side of them, and treated those interfaces as frozen artefacts that could be changed only by mutual agreement. The pay-off is that the two human workstreams — a backend workstream covering the orchestrator, database, portal, and event pipeline, and an

AI/LLM workstream covering context extraction, vector search, re-ranking, response generation, and engagement judging — were able to develop in parallel against typed mocks of one another and integrate in a single short sitting rather than in a protracted debugging phase.

The contract is expressed as a small set of Python dataclasses and asynchronous function signatures. Four data types carry almost the entire load. A `Turn` is an immutable record of one message in a conversation, tagged with its role, timestamp, and the identifier of any ad that was shown alongside it. A `ContextObject` is the structured projection of a user's most recent turn, carrying an enumerated `intent`, a list of topics and entities, a sentiment label, a purchase-signal scalar in $[0, 1]$, and an `ad_receptivity` enum whose `none` value short-circuits the matching pipeline entirely for sensitive conversations. An `Ad` is the unit of inventory, carrying targeting metadata, creative text, brand-safety tags, a bid in CPM, a remaining budget, and two optional fields — `similarity` and `score` — that are populated in sequence by the vector search and the re-ranker. A `ResponseObject` is what the orchestrator returns to the frontend: a response text, a boolean indicating whether an ad was included, and the identifier of the ad used if so. The enums (`Intent`, `AdReceptivity`) are part of the contract rather than free-form strings so that a typo on one side of the seam becomes a type error at import time rather than a silent mismatch at runtime.

Four asynchronous function signatures then pin down the call graph between the workstreams. `extract_context(message, history)` $\rightarrow$ `ContextObject` is owned by the AI workstream and called by the orchestrator once per user turn. `match_ads(context, session_id)` $\rightarrow$ `list[Ad]` is also owned by the AI workstream and encapsulates the embedding, vector search, and re-ranking stages behind a single entry point, so that the backend need not know whether matching is performed by a single model call or a multi-stage pipeline. `generate_response(message, history, ad)`$\rightarrow$ `ResponseObject` is the main model call that weaves the selected ad (if any) into a helpful answer. Finally, `judge_engagement(ad, prev_assistant_turn, user_turn)` $\rightarrow$ `EngagementVerdict` is the post-hoc judge described in Section 5.4.2. Everything that is not in this list — database access, Redis session state, event writes, portal CRUD, budget pacing — is owned unambiguously by the backend workstream and is invisible to the AI workstream except through these four functions.

The division of labour has two consequences worth naming. First, either side can be replaced wholesale without touching the other: swapping the matching pipeline for a learned end-to-end ranker, or swapping the Postgres event store for a streaming pipeline, requires changes only inside one workstream's boundary. Second, testing becomes local. The backend workstream tests against a deterministic stub of the AI functions that returns canned `ContextObject` and `Ad` values; the AI workstream tests against an in-memory fake of the session store and event writer. Neither side needs the other to be running in order to validate its own invariants.

5.7 Quality Assurance and Evaluation Framework

Evaluating a conversational ad system is harder than evaluating either a chat model or a classical ad ranker in isolation, because the quality of a served ad is a joint function of its relevance to the turn, its compatibility with the surrounding assistant prose, and its effect on the user's subsequent behaviour. We therefore adopt a layered evaluation framework in which each component is held to a local correctness criterion and the system as a whole is judged against a small number of end-to-end metrics that reflect the product's stated value proposition.

At the component level, three properties are checked continuously. Context extraction is evaluated for *schema validity* and *label stability*: the Haiku-tier extractor must return a `ContextObject` that parses against the dataclass schema on every call, and its labels on a fixed regression set of fifty hand- annotated turns must remain stable across prompt revisions. A failure of the first property is caught at parse time and triggers a fallback to a conservative default; a failure of the second is caught by a nightly job that diffs the extractor's output against the regression set and opens an alert if agreement drops below a threshold. Vector search is evaluated for *recall at 20* on a small synthetic set of query–ad pairs constructed from the inventory's own product descriptions, which gives a cheap, self-generated benchmark that automatically tracks changes in the embedding model or the inventory. The re-ranker is evaluated by *rank correlation* between its output ordering and the ordering induced by the unmodulated similarity score, with the expectation that the two should diverge in interpretable ways — budget-exhausted ads should fall, high purchase-signal sessions should see higher-bid ads rise — and that the divergence itself is what the unit tests assert.

At the end-to-end level, the system is judged on three metrics. *Engagement rate*, the fraction of served ads whose subsequent user turn is flagged as engaged by the judge of Section 5.4.2, is the headline quality metric and is the one that the feedback loop optimises against. *Click-through rate* is reported alongside it as a secondary metric, mainly for comparability with conventional ad analytics. *Injection appropriateness* is the rate at which ads are served into turns whose `ad_receptivity` is `high` or `medium` and suppressed on turns whose receptivity is `low` or `none`; this metric exists to catch regressions in the sensitivity classifier independently of whether the served ads are themselves good, because an inappropriately-placed ad is a product failure even if the user happens to engage with it.

We deliberately do not use offline A/B testing as a primary evaluation tool. The feedback loop described in Section 5.4.3 means that serving decisions on one turn shape context on subsequent turns within the same session, which violates the independence assumption that conventional A/B tests rely on. Instead, we rely on the regression sets and rank-correlation tests above to catch local regressions, and on the headline engagement rate to track global drift. A principled bandit-style evaluation that respects the within-session dependency structure is sketched in Section 7.4 as the natural successor to the current framework.

5.8 Error Handling and Graceful Degradation

The serving path touches three external systems that can each fail independently — a Postgres instance with a pgvector extension, a Redis instance holding session state, and one or more remote LLM endpoints — and any production deployment must answer the question of what the user sees when one of them is unreachable or slow. Our guiding principle is that *the chat response is the product*, and that every component in the ad pipeline is expendable in the service of delivering a timely, coherent answer to the user. No failure in the ad pipeline should ever degrade the conversational experience, and no ad should ever be served on the back of partial or inconsistent data.

This principle is realised as a small number of explicit fallback paths, each triggered by a specific class of failure. A failure of the context extractor — whether a timeout, a malformed

JSON response, or a schema validation error — causes the orchestrator to substitute a default `ContextObject` whose `ad_receptivity` is set to `none`. This value is the one the matching pipeline interprets as "do not serve an ad on this turn," so the effect of the failure is silent suppression: the user receives a normal assistant response with no sponsored content, and an error event is written to the log for later inspection. A failure of the vector search, whether due to a Postgres outage or an empty candidate set, is treated identically: the matching function returns an empty list, the orchestrator observes that no candidates are available, and the response-generation call is issued without any ad in its prompt. A failure of Redis is handled by treating the session as if it were new; the current turn is extracted and matched in isolation, and the adaptive within-session loop is skipped for that turn only. In all three cases the system degrades to a strictly weaker but still correct version of itself, rather than surfacing an error to the user or blocking the response.

The main-model response generation call is the one external dependency that cannot be silently elided, because there is no response without it. For this call we adopt a bounded-retry policy with exponential backoff and a hard deadline of a few seconds, beyond which the orchestrator returns a generic error message to the frontend and logs the incident. Ad-related state is never committed for a turn whose response generation failed: no impression event is written, no session state is updated, and no budget is debited. This preserves the invariant that every impression in the events table corresponds to an ad the user actually saw.

A second class of failures, subtler than outright outages, concerns partial inconsistency between components. The portal, for instance, can mark a campaign as paused between the moment an ad was selected by the re-ranker and the moment the response is generated; a campaign's daily budget can be exhausted by a concurrent request in the same window; an ad's embedding can be regenerated in place after the candidate list was assembled. We address these races with a final-check pattern: immediately before the response-generation call, the orchestrator re-reads the selected ad's status, budget, and version from the database inside a single short transaction and aborts the injection if any invariant has been violated since selection. The cost of this re-read is one indexed point lookup per turn, which is negligible next to the model call it precedes, and the benefit is that the events table never contains an impression for an ad that was paused, out of budget, or stale at the instant of serving. Taken together, these mechanisms give the system a clear failure contract: ads may silently disappear, but the conversational experience does not, and the analytics layer never reports on events that did not faithfully happen.

6 Illustrative Examples

6.1 Example 1: Product Research Conversation

6.2 Example 2: Sensitive Conversation (Ad Suppression)

6.3 Example 3: Irrelevant Ad Rejection

6.4 Example 4: Multi-Dimensional Engagement Follow-Up

6.5 Example 5: Adaptive Context Optimisation Loop

6.6 Example 6: Image Generation with Product Placement

6.7 Example 7: Campaign Lifecycle and Budget Exhaustion

7 Conclusions

7.1 Summary of Contributions

7.2 Workstream Contributions

7.3 Limitations

7.4 Future Work

7.5 Reflections

References

[1] Anthropic, *The Claude 3 model family: Opus, Sonnet, Haiku*, Introduced three model tiers with context windows up to 200K tokens, 2024. [Online]. Available: https://www.anthropic.com/news/claude-3-family.

[2] Apple Inc., *App tracking transparency*, Introduced in iOS 14.5, requiring apps to obtain user permission before tracking across apps and websites, 2021. [Online]. Available: https://developer.apple.com/documentation/apptrackingtransparency.

[3] Y. Bai, S. Kadavath, S. Kundu, A. Askell, J. Kernion, A. Jones, A. Chen, A. Goldie, A. Mirhoseini, C. McKinnon, et al., *Constitutional AI: Harmlessness from AI feedback*, 2022. arXiv: 2212.08073 [cs.CL].

[4] S. K. Balasubramanian, J. A. Karrh, and H. Patwardhan, "Audience response to product placements: An integrative framework and future research agenda," *Journal of Advertising*, vol. 35, no. 3, 2006.

[5] J. Betker, G. Goh, L. Jing, T. Brooks, J. Wang, L. Li, et al., "Improving image generation with better captions," *OpenAI Technical Report*, 2023.

[6] T. Brooks, A. Holynski, and A. A. Efros, "Instructpix2pix: Learning to follow image editing instructions," in *CVPR*, 2023.

[7] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al., "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 1877–1901.

[8] C. Campbell and N. J. Evans, "The role of a companion banner and sponsorship transparency in recognizing and evaluating article-style native advertising," *Journal of Interactive Marketing*, vol. 43, pp. 17–32, 2018.

[9] C. T. Carr and R. A. Hayes, "The effect of disclosure of third-party influence on an opinion leader's credibility and electronic word of mouth in two-step flow," *Journal of Interactive Advertising*, vol. 14, no. 1, pp. 38–50, 2014. DOI: 10.1080/15252019.2014.909296.

[10] L. Chen, M. Zaharia, and J. Zou, *FrugalGPT: How to use large language models while reducing cost and improving performance*, 2023. arXiv: 2305.05176 [cs.LG].

[11] S. van Dam and E. A. van Reijmersdal, "Disclosure-driven recognition of native advertising: A test of two competing mechanisms," *Journal of Interactive Advertising*, vol. 22, no. 3, pp. 261–276, 2022. DOI: 10.1080/15252019.2022.2146991.

[12] eMarketer, *Programmatic digital display advertising spending worldwide, 2022–2026*, Programmatic advertising accounted for 84% of global digital display spending in 2022, 2022.

[13] European Parliament and Council of the European Union, "Regulation (EU) 2016/679 of the European Parliament and of the Council (general data protection regulation)," *Official Journal of the European Union*, vol. L 119, pp. 1–88, 2016.

[14] Federal Trade Commission, *Guides concerning the use of endorsements and testimonials in advertising*, Revised in June 2023 to address social media influencers, virtual endorsers, and AI-generated content, 2023. [Online]. Available: https://www.ftc.gov/legal-library/browse/federal-register-notices/16-cfr-part-255-guides-concerning-use-endorsements-testimonials-advertising.

[15] C. Fernando, D. Banarse, H. Michalewski, S. Osindero, and T. Rocktäschel, "Promptbreeder: Self-referential self-improvement via prompt evolution," *arXiv preprint arXiv:2309.16797*, 2023.

[16] Google, *Introducing a new era of AI-powered ads with Google*, Announced integration of Search and Shopping ads within the Search Generative Experience (SGE), 2023. [Online]. Available: https://blog.google/products/ads-commerce/ai-powered-ads-google-marketing-live/.

[17] Google, *Google AI Mode: Ads in AI-powered search*, Extended ad placements into AI Mode, a conversational search interface powered by Gemini 2.0, 2025.

[18] Google DeepMind, *Gemini: A family of highly capable multimodal models*, 2024. arXiv: 2312.11805 [cs.CL].

[19] J. Gu, X. Jiang, Z. Shi, H. Tan, X. Zhai, C. Xu, L. Li, J. Wen, L. Liu, and Q. Li, *A survey on LLM-as-a-judge*, 2024. arXiv: 2411.15594 [cs.CL].

[20] HotWired, *The first banner ad on the web*, The first banner advertisement, placed by AT&T on HotWired.com, reportedly achieved a click-through rate exceeding 44%, 1994.

[21] Z. Jing, Y. Su, and Y. Han, *When large language models meet vector databases: A survey*, 2024. arXiv: 2402.01763 [cs.DB].

[22] J. Kang and G. Hustvedt, "Building trust between consumers and corporations: The role of consumer perceptions of transparency and social responsibility," *Journal of Business Ethics*, vol. 125, no. 2, pp. 253–265, 2014. DOI: 10.1007/s10551-013-1916-7.

[23] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense passage retrieval for open-domain question answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6769–6781.

[24] A. Katz, *Pgvector: Open-source vector similarity search for PostgreSQL*, 2023. [Online]. Available: https://github.com/pgvector/pgvector.

[25] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[26] L. Li, W. Chu, J. Langford, and R. E. Schapire, "A contextual-bandit approach to personalized news article recommendation," in *Proceedings of the 19th International Conference on World Wide Web (WWW)*, 2010.

[27] S. Mehri and M. Eskénazi, "Unsupervised evaluation of interactive dialog with DialoGPT," in *Proceedings of the 21st Annual Meeting of the Special Interest Group on Discourse and Dialogue (SIGdial)*, 2020, pp. 225–235.

[28] Microsoft Advertising, *Microsoft introduces conversational ads for Bing Chat*, New ad formats designed specifically for chat-based search experiences, Sep. 2023.

[29] Microsoft Advertising, *Transforming search and advertising with generative AI*, Announced Conversational Ads and Compare & Decide Ads for Bing Chat, 2023. [Online]. Available: https://about.ads.microsoft.com/en/blog/post/september-2023/transforming-search-and-advertising-with-generative-ai.

[30] Nielsen, *Native advertising: Perception and disclosure*, 79% of users identified content as advertising when labelled "Advertisement," compared to 48% for "Presented by", 2014.

[31] OpenAI, *GPT-4 technical report*, 2023. arXiv: 2303.08774.

[32] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al., "Training language models to follow instructions with human feedback," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 27 730–27 744.

[33] E. A. van Reijmersdal, P. C. Neijens, and E. G. Smit, "A new branch of advertising: Reviewing factors that influence reactions to product placement," *Journal of Advertising Research*, vol. 49, no. 4, 2009.

[34] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *CVPR*, 2022.

[35] N. Ruiz, Y. Li, V. Jampani, Y. Pritch, M. Rubinstein, and K. Aberman, "Dreambooth: Fine tuning text-to-image diffusion models for subject-driven generation," in *CVPR*, 2023.

[36] C. Saharia et al., "Photorealistic text-to-image diffusion models with deep language understanding," *NeurIPS*, 2022.

[37] P. Sahoo, A. K. Singh, S. Saha, V. Jain, S. Mondal, and A. Chadha, *A systematic survey of prompt engineering in large language models: Techniques and applications*, 2024. arXiv: 2402.07927 [cs.CL].

[38] S. Schulhoff, M. Ilie, N. Balepur, K. Kahadze, A. Liu, C. Si, Y. Li, A. Gupta, H. Han, S. Schulhoff, et al., *The prompt report: A systematic survey of prompting techniques*, 2024. arXiv: 2406.06608 [cs.CL].

[39] A. See, S. Roller, D. Kiela, and J. Weston, "What makes a good conversation? how controllable attributes affect human judgments," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 2019, pp. 1702–1723.

[40] Statista, *Google: Advertising revenue worldwide 2001–2024*, In 2024, Google's advertising revenue reached approximately $264.59 billion, 2024. [Online]. Available: https://www.statista.com/statistics/266249/advertising-revenue-of-google/.

[41] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, et al., *LLaMA: Open and efficient foundation language models*, 2023. arXiv: 2302.13971 [cs.CL].

[42] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[43] L. Wang, N. Yang, X. Huang, B. Jiao, L. Yang, D. Jiang, R. Majumder, and F. Wei, *Text embeddings by weakly-supervised contrastive pre-training*, 2022. arXiv: 2212.03533 [cs.CL].

[44] J. Wei, Y. Tay, R. Bommasani, C. Raffel, B. Zoph, S. Borgeaud, D. Yogatama, M. Bosma, D. Zhou, D. Metzler, et al., "Emergent abilities of large language models," *Transactions on Machine Learning Research*, 2022.

[45] B. W. Wojdynski and N. J. Evans, "Going native: Effects of disclosure position and language on the recognition and evaluation of online native advertising," *Journal of Advertising*, vol. 45, no. 2, pp. 157–168, 2016.

[46] S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff, *C-Pack: Packaged resources to advance general Chinese embedding*, 2023. arXiv: 2309.07597 [cs.CL].

[47] S. Yuan, A. Z. Abidin, M. Sloane, and J. Wang, "Real-time bidding for online advertising: Measurement and analysis," in *Proceedings of the Seventh International Workshop on Data Mining for Online Advertising (ADKDD)*, ACM, 2013. DOI: 10.1145/2501040.2501980.

[48] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[49] Y. Zhou, A. I. Muresanu, Z. Han, K. Paster, S. Pitis, H. Chan, and J. Ba, "Large language models are human-level prompt engineers," *International Conference on Learning Representations (ICLR)*, 2023.

A Database Schema

B API Endpoint Specifications

C Prompt Templates

D Context Extraction Test Cases

E Response Synthesis Evaluation Set

F Session State Schema

G Project Timeline

Table 2. Project timeline by workstream.

Week	Backend Workstream	AI/LLM Workstream	Milestone
1	Orchestrator skeleton, Redis, Postgres + pgvector schema	Context extraction prompt design + module	Infrastructure foundation
2	Ad CRUD API, bulk ingestion, re-ranking logic	Embedding functions, retrieval quality testing	Ad matching pipeline ready
3	Click tracking, event logging, tracking link generator	Response synthesis prompt engineering + module	Core pipeline complete
4	Analytics API, frontend integration, end-to-end wiring	Eval set, quality tuning, engagement classifier	End-to-end integration
5	Error handling, monitoring, hardening	LLM-as-judge evaluator, attention model start	Demo-ready MVP